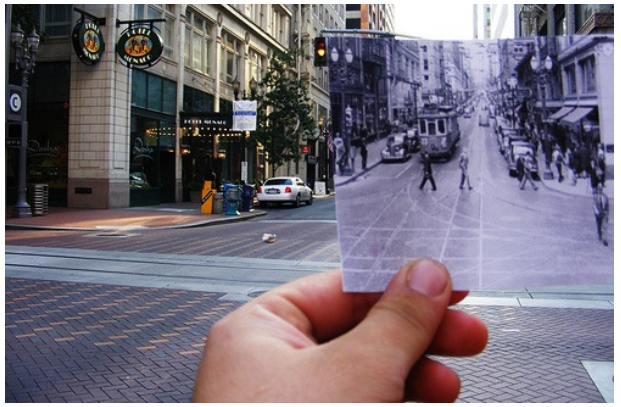
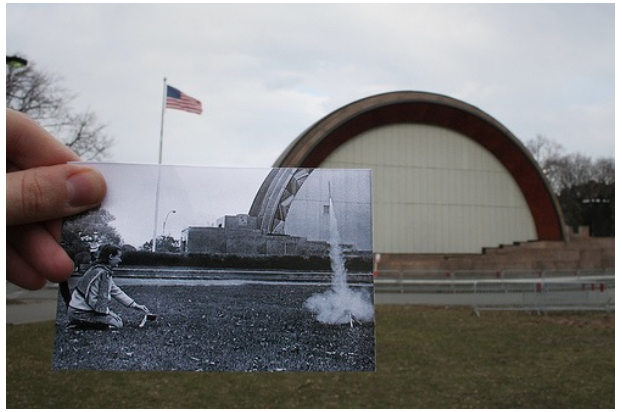
Image Alignment

Slides are from Svetlana Lazebnik

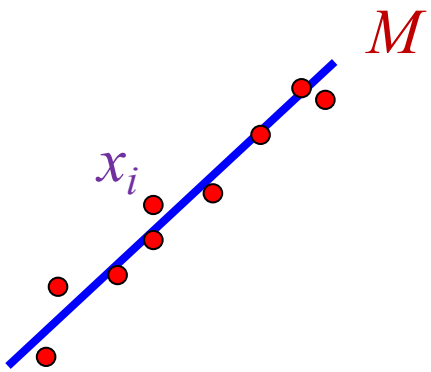




Source

Alignment: Fitting of Transformations

- Previously: fitting a model to features in one image

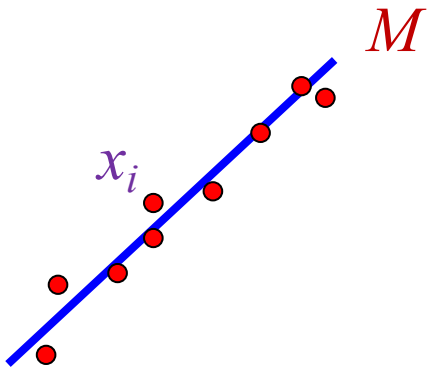


- Given: points $x_1, \ldots, x_n$
- Find: model M that minimizes

$$\sum_i \text{residual}(x_i, M)$$

Alignment: Fitting of Transformations

- Previously: fitting a model to features in one image

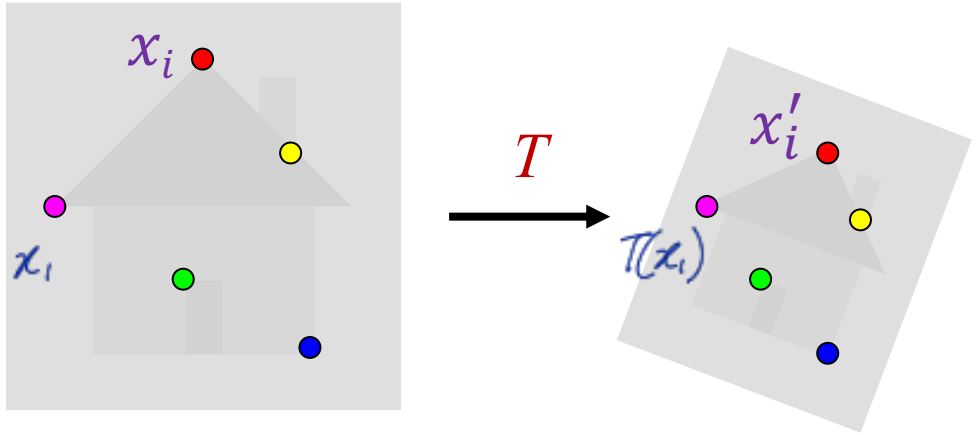


- Given: points $x_1, \ldots, x_n$
- Find: model M that minimizes

$$\sum_i \text{residual}(x_i, M)$$

$(y_i - \hat{y}_i)^2$

- Alignment: fitting a model to a transformation between pairs of features (*matches*) in two images



- Given: matches $(x_1, x_1'), \ldots, (x_n, x_n')$ — usually matrices
- Find: transformation T that minimizes

$$\sum_i \text{residual}(T(x_i), x_i')$$

true transformed image

same idea

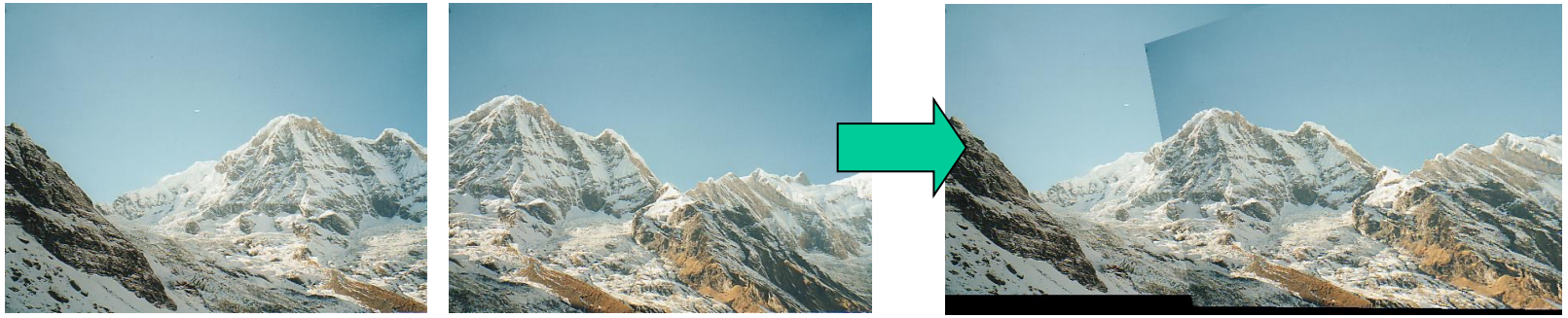
predicted transform

Alignment: Overview

- Motivation
- Fitting of transformations
  - Affine transformations
  - Homographies
- Robust alignment
  - Descriptor-based feature matching
  - RANSAC
- Large-scale alignment
  - Inverted indexing
  - Vocabulary trees

we try to transform image 1 to 2 such that $x_i = T(x_i)$ as much as possible

Alignment Applications: Panorama Stitching



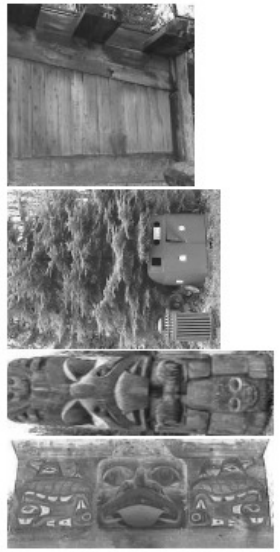
http://matthewalunbrown.com/autostitch/autostitch.html

Aligning Contemporary and Historic Images

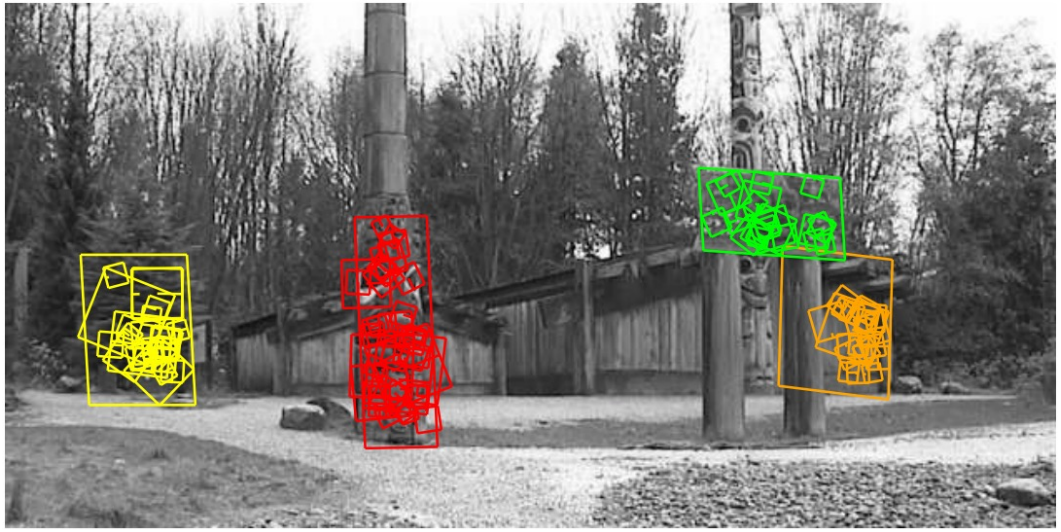
Source

Alignment Applications: Instance Recognition

Model images



Test image



David G. Lowe. Distinctive image features from scale-invariant keypoints. *IJCV* 60 (2), pp. 91-110, 2004

Alignment Applications: Instance Recognition

T. Weyand and B. Leibe, Visual landmark recognition from Internet photo collections: A large-scale evaluation, CVIU 2015

Alignment Applications: Large-Scale Reconstruction

Colosseum: 2,097 images, 819,242 points

Trevi Fountain: 1,935 images, 1,055,153 points

Pantheon: 1,032 images, 530,076 points

Hall of Maps: 275 images, 230,182 points

S. Agarwal et al. Building Rome in a Day. ICCV 2009

Feature-Based Alignment

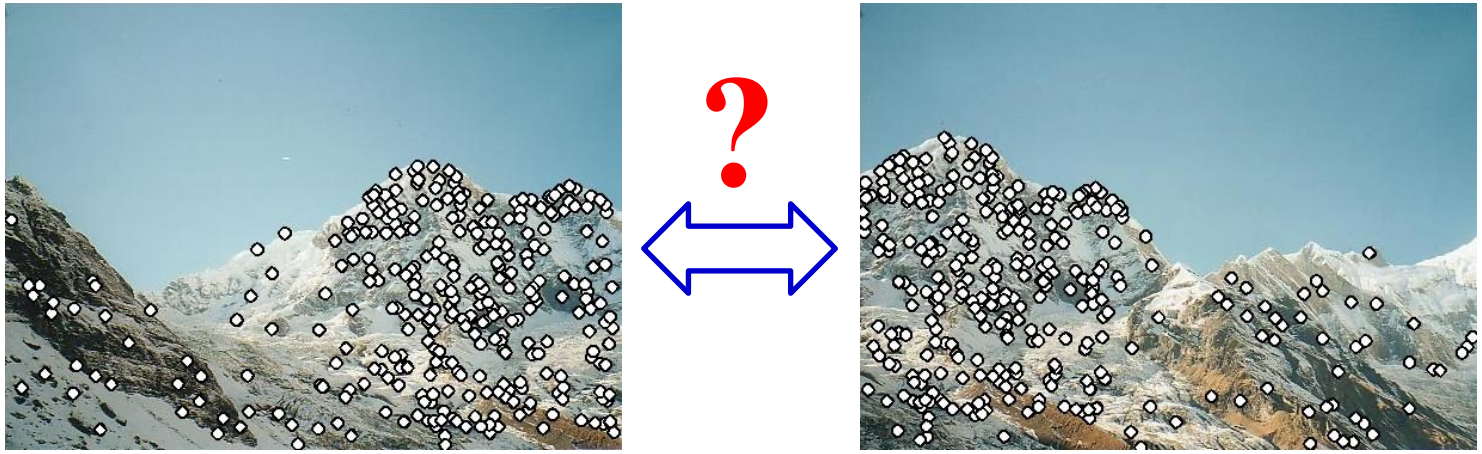
first need to find x_i and x_i' pairs

Find a set of of feature matches that agree in terms of:

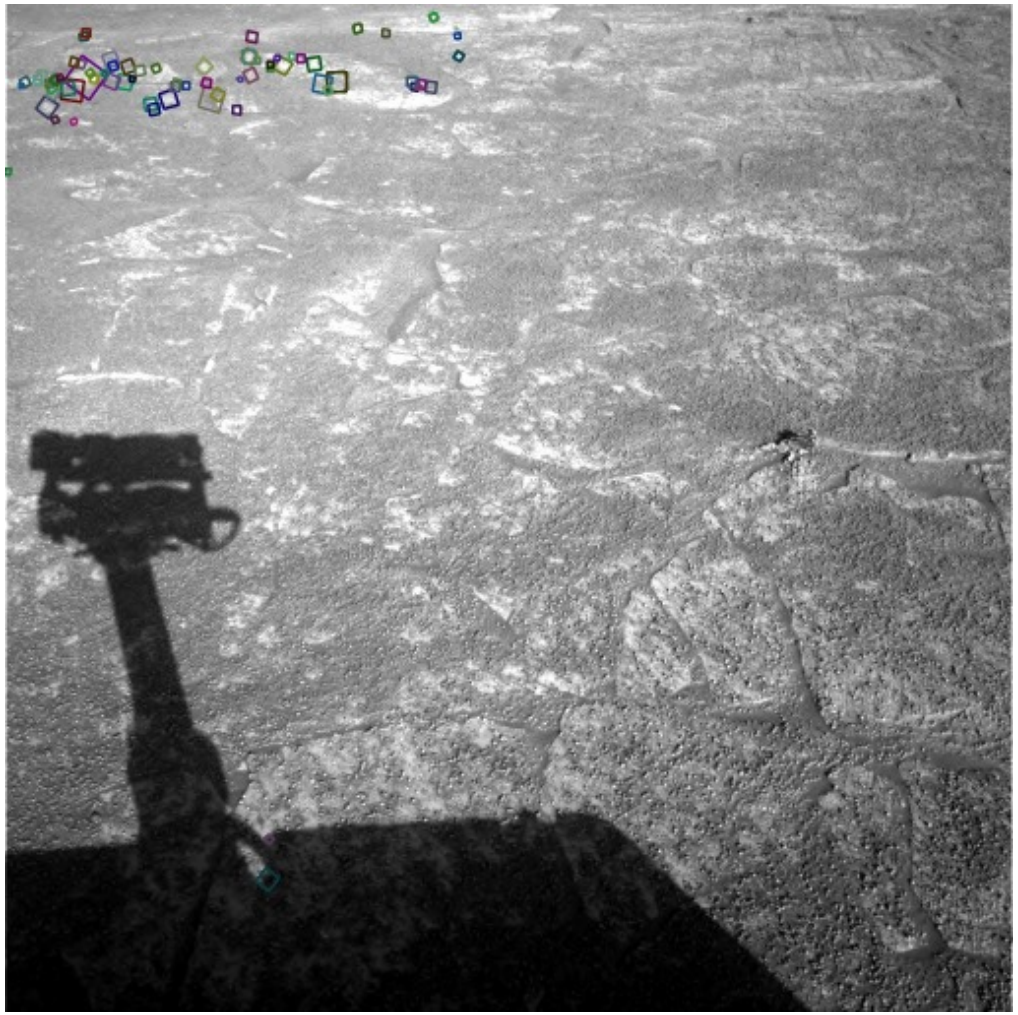
a) Local appearance — x_i and x_i' matches

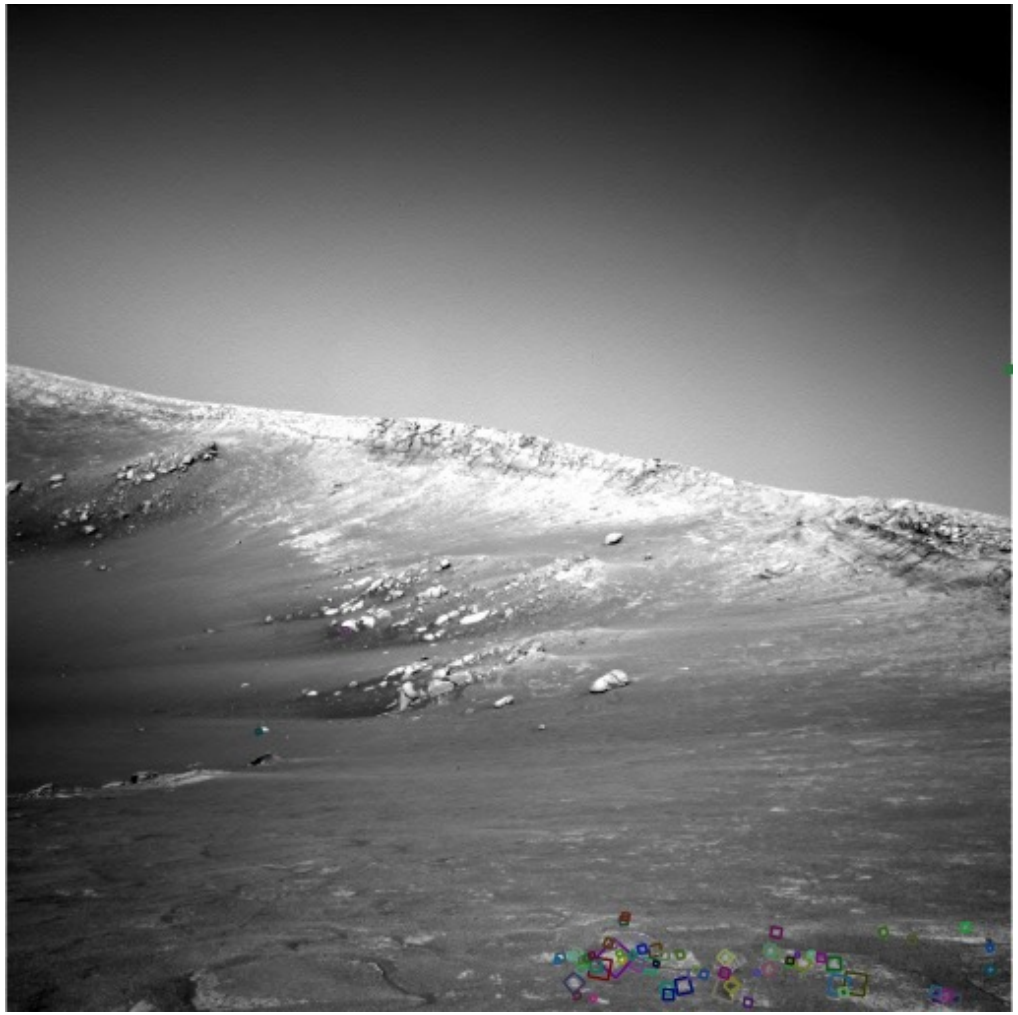
b) Geometric configuration — A transform for all pairs

Then find T such that $T(x_i) \approx x_i'$



Feature-Based Alignment *Really Works*!





Source: N. Snavely

Feature-Based Alignment *Really Works*!

Source: N. Snavely

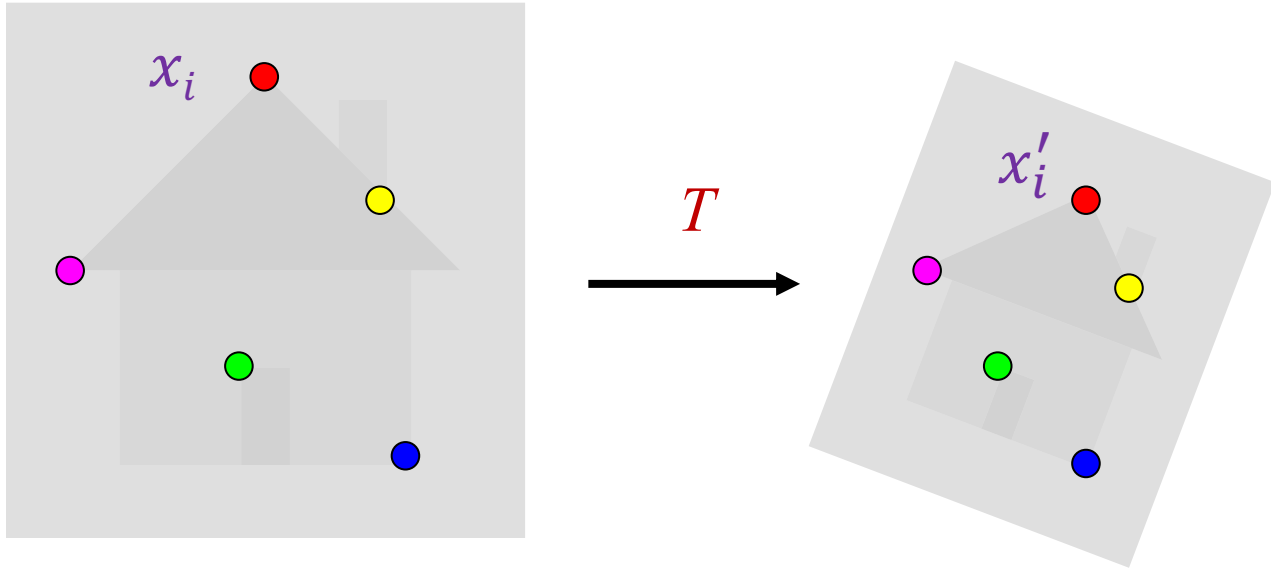
Alignment: Overview

- Motivation
- Fitting of transformations
  - Affine transformations
  - Homographies

Alignment: Fitting of transformations

- Given: matches $(x_1, x_1'), \ldots, (x_n, x_n')$
- Find: transformation T that minimizes

$$\sum_i \text{residual}(T(x_i), x_i')$$



Hartley and Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., 2004, p. 44

Class I Isometries

displacement + rotation

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{bmatrix} \epsilon\cos\theta & -\sin\theta & t_x \\ \epsilon\sin\theta & \cos\theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

outputs; inputs; θ - rotation; ϵ - rotation direction; t - displacement

$$\epsilon = \pm 1$$

$$\mathbf{x}' = \mathbf{H}_{\mathrm{E}}\mathbf{x} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^{\mathsf{T}} & 1 \end{bmatrix} \mathbf{x}$$

rotation part; translation part; $\epsilon=+1$; $\epsilon=-1$

$$\mathbf{R}^{\mathsf{T}}\mathbf{R} = \mathbf{R}\mathbf{R}^{\mathsf{T}} = \mathbf{I}$$

orthonormal

Class II Similarity transformations

displacement + rotation + scaling

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{bmatrix} s\cos\theta & -s\sin\theta & t_x \\ s\sin\theta & s\cos\theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

s - scaling factor

$$\mathbf{x}' = \mathbf{H}_{\mathrm{S}}\mathbf{x} = \begin{bmatrix} s\mathbf{R} & \mathbf{t} \\ \mathbf{0}^{\mathsf{T}} & 1 \end{bmatrix} \mathbf{x}$$

translation; scaling + rotation

Class III Affine transformations

translation + angled view (parallel lines)

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

affine transformation

$$\mathbf{x}' = \mathbf{H}_{\mathrm{A}}\mathbf{x} = \begin{bmatrix} \mathbf{A} & \mathbf{t} \\ \mathbf{0}^{\mathsf{T}} & 1 \end{bmatrix} \mathbf{x}$$

A 2×2 non-singular matrix

Class IV Projective transformations

also called homography

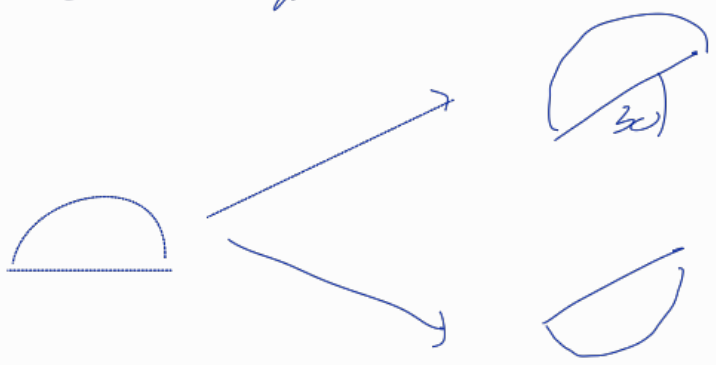
translation + angled view (warped non-parallel lines)

$$\mathbf{x}' = \mathbf{H}_{\mathrm{P}}\mathbf{x} = \begin{bmatrix} \mathbf{A} & \mathbf{t} \\ \mathbf{v}^{\mathsf{T}} & v \end{bmatrix} \mathbf{x}$$

Specified by eight parameters

Hartley and Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., 2004, p. 42-44

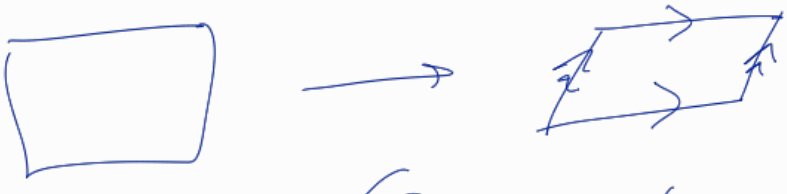
① rotate + displacement



$\varepsilon = 1$ Rotate

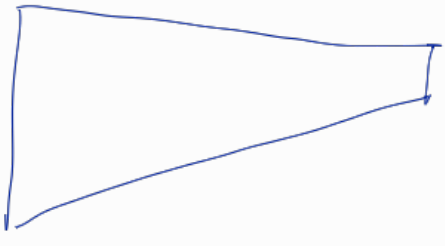
$\varepsilon = -1$ Rotate + reflection

② rotate + displacement + scale

③ affine 

parallel lines (& small objects)

④ projective ⇒ non parallel

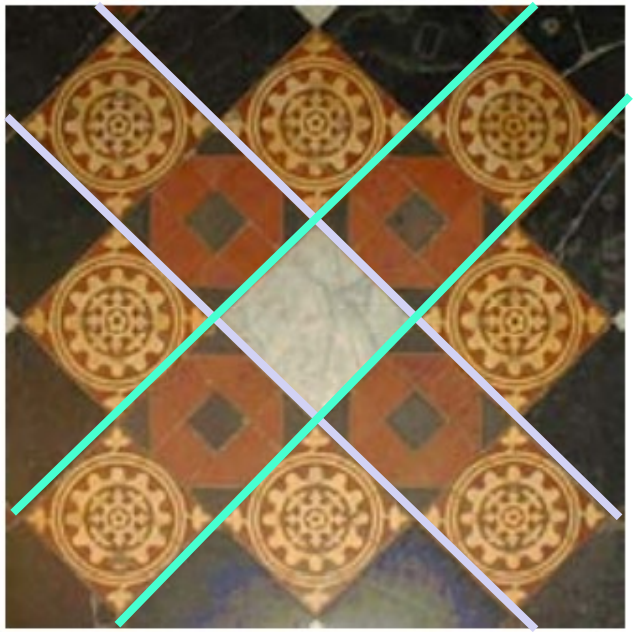


Rhombus (eg: real container)

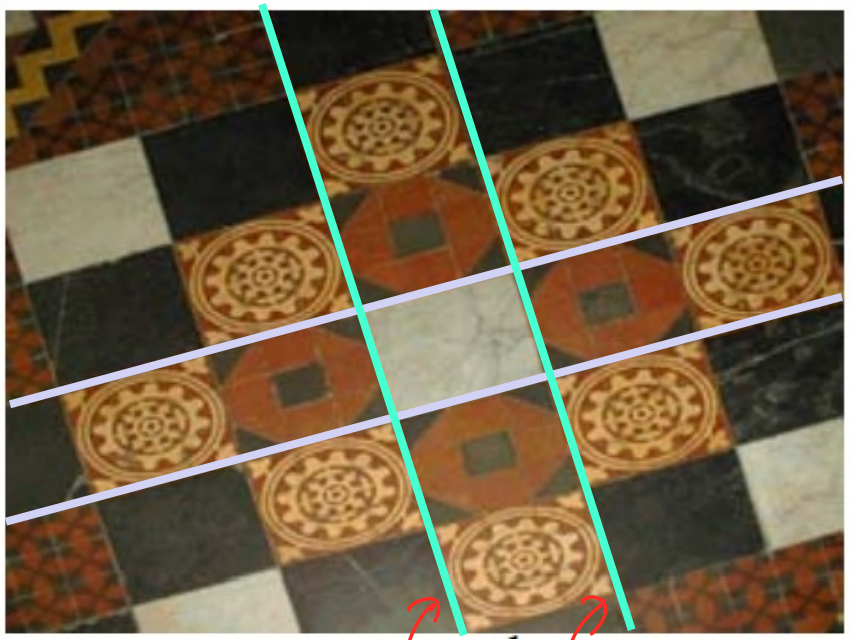
also known as homography

keep in mind we $\boxed{\lambda x' = H_p x}$

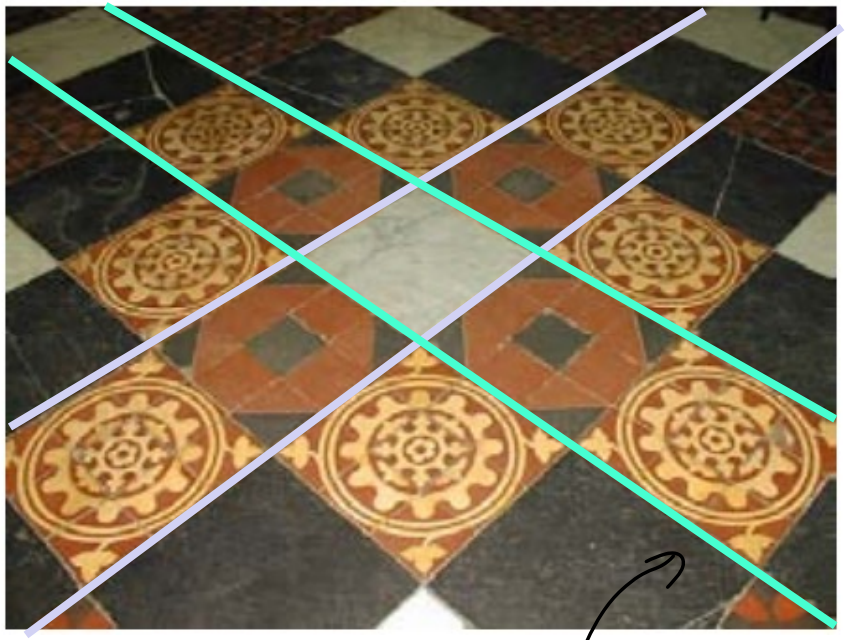
2-D Transformations



Similarity



Affine

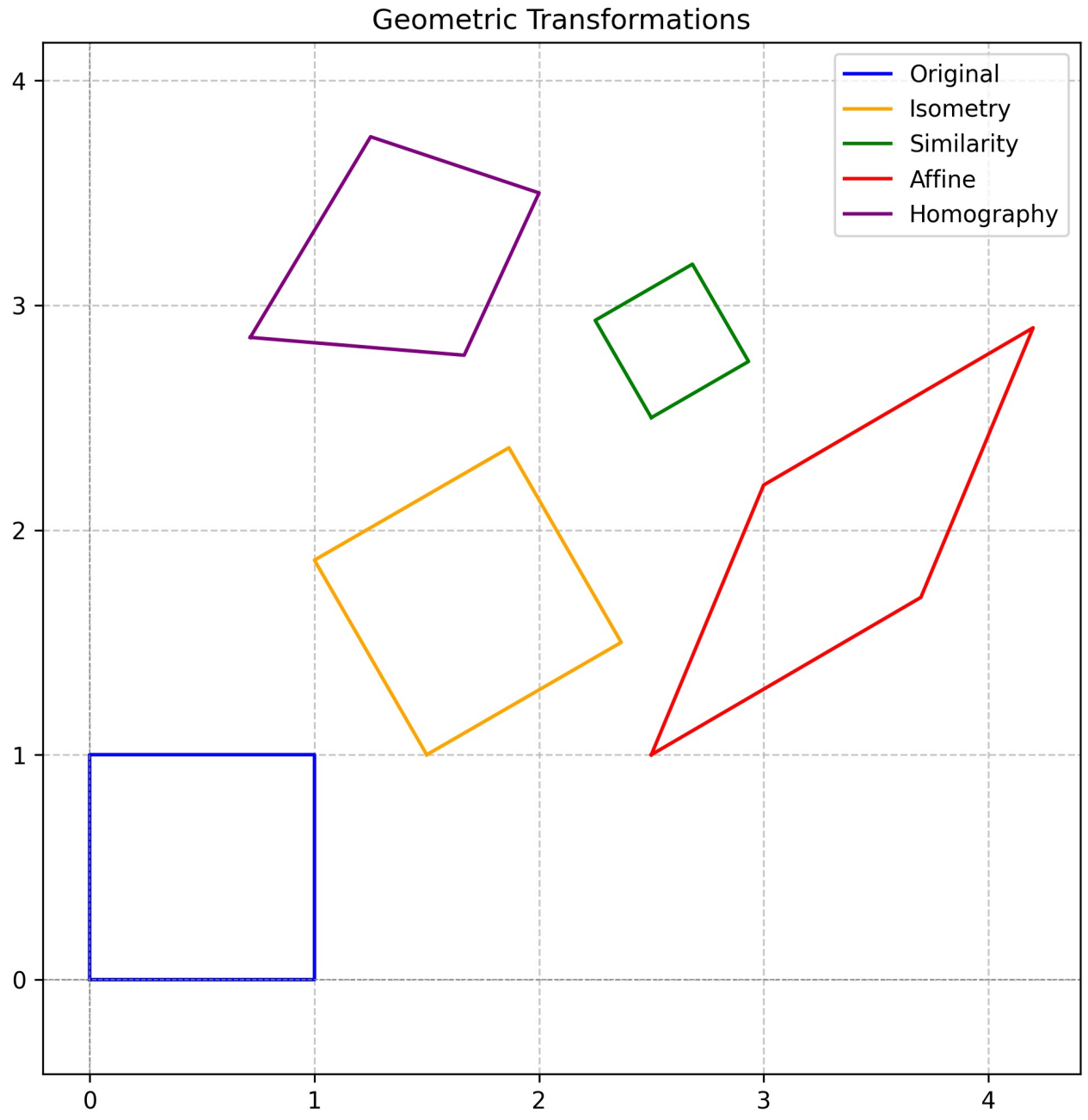


Projective

Decomposition of a projective transformation

$$\mathbf{H} = \mathbf{H}_S \mathbf{H}_A \mathbf{H}_P = \begin{bmatrix} s\mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \begin{bmatrix} \mathbf{K} & \mathbf{0} \\ \mathbf{0}^T & 1 \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{v}^T & v \end{bmatrix} = \begin{bmatrix} \mathbf{A} & \mathbf{t} \\ \mathbf{v}^T & v \end{bmatrix}$$

Hartley and Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., 2004, p. 44



Although in homograph t= [0.5, 2] image is not translated there. This is because of v=0.7 in Hp

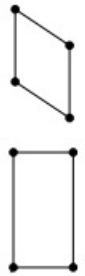
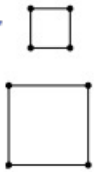
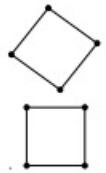
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H_p \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

x', y' are dummy parameters use $\frac{x'}{z}, \frac{y'}{z}$ to get actual values

```python
# Transformation 1: Isometry (Rotation + Translation)
theta = np.pi / 6  # Rotation angle (30 degrees)
t = np.array([1.5, 1])  # Translation vector
H_isometry = np.array([
    [np.cos(theta), -np.sin(theta), t[0]],
    [np.sin(theta), np.cos(theta), t[1]],
    [0, 0, 1],
])

# Transformation 2: Similarity (Rotation + Scaling + Translation)
s = 0.5  # Scaling factor
t = np.array([2.5, 2.5])  # Translation vector
H_similarity = np.array([
    [s * np.cos(theta), -s * np.sin(theta), t[0]],
    [s * np.sin(theta), s * np.cos(theta), t[1]],
    [0, 0, 1],
])

# Transformation 3: Affine (Skewing + Scaling + Translation)
t = np.array([2.5, 1])  # Translation vector
H_affine = np.array([
    [1.2, 0.5, t[0]],
    [0.7, 1.2, t[1]],
    [0, 0, 1],
])

# Transformation 4: Homography (Perspective)
t = np.array([0.5, 2])  # Translation vector
H_homography = np.array([
    [1, 0.5, t[0]],
    [0.5, 1, t[1]],
    [0.2, 0.1, 0.7],
])

# List of transformations, labels, and colors
transformations = [H_isometry, H_similarity, H_affine, H_homography]
labels = ["Isometry", "Similarity", "Affine", "Homography"]
colors = ["orange", "green", "red", "purple"]

# Plot the transformations
plot_transformation(X, transformations, labels, colors)
```

Translation is set [0.5,2] to find true translation we need to solve $\begin{bmatrix} x' \\ y' \\ z \end{bmatrix} = H_p \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$ and get x/z, y/z

2D Transformation Models

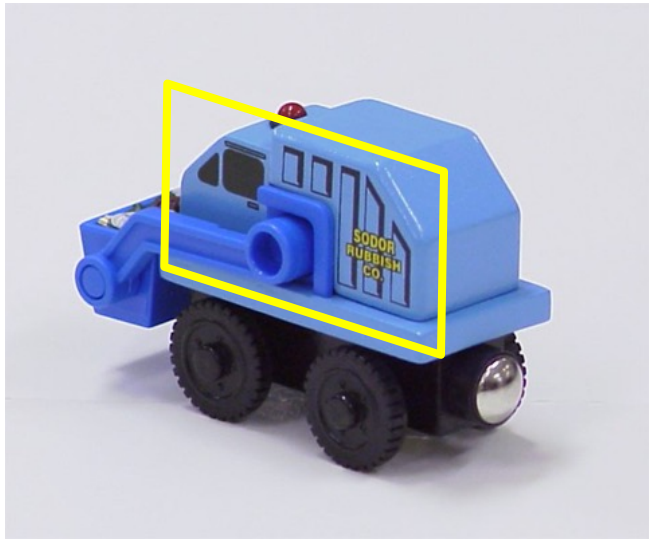
Group	Matrix	Distortion	Invariant properties
Projective 8 dof	$\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$		Concurrency, collinearity, **order of contact**: intersection (1 pt contact); tangency (2 pt contact); inflections (3 pt contact with line); tangent discontinuities and cusps. cross ratio (ratio of ratio of lengths).
Affine 6 dof	$\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$		Parallelism, ratio of areas, ratio of lengths on collinear or parallel lines (e.g. midpoints), linear combinations of vectors (e.g. centroids). The line at infinity, $\mathbf{l}_\infty$.
Similarity 4 dof	$\begin{bmatrix} sr_{11} & sr_{12} & t_x \\ sr_{21} & sr_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$		Ratio of lengths, angle. The circular points, $\mathbf{I}, \mathbf{J}$ (see section 2.7.3).
Euclidean 3 dof	$\begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$		Length, area

Hartley and Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., 2004, p. 44

Let's Start with Affine Transformations

Need 6 parameters

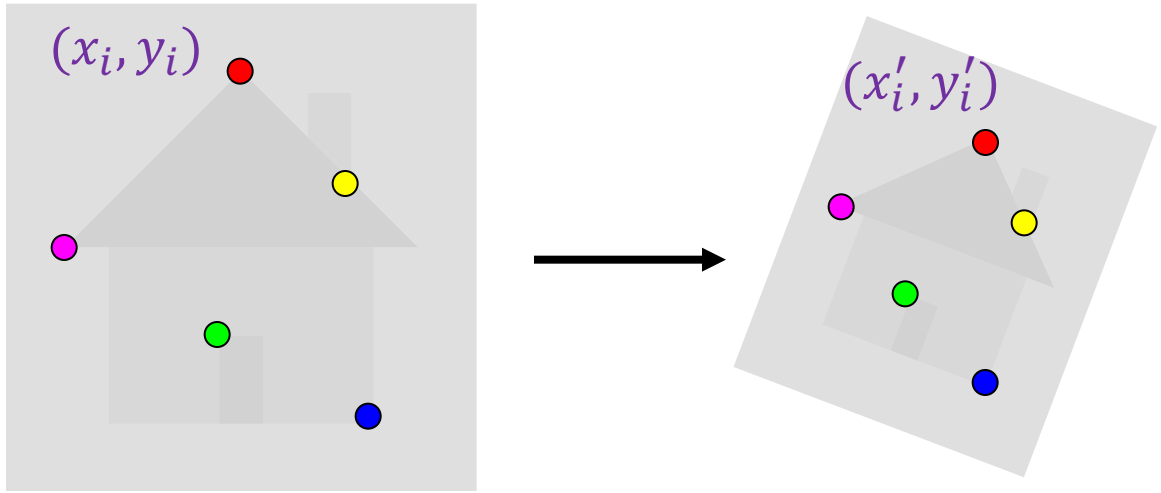
- Simple fitting procedure: linear least squares
- Approximates viewpoint changes for roughly planar objects and roughly orthographic cameras
- Can be used to initialize fitting for more complex models



Fitting an Affine Transformation

- Assume we know the correspondences, how do we get the transformation?

$$\begin{bmatrix} x_i' \\ y_i' \\ 1 \end{bmatrix} = \begin{bmatrix} m_1 & m_2 & t_1 \\ m_3 & m_4 & t_2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} \Rightarrow \begin{aligned} x_i' &= m_1 x_i + m_2 y_i + t_1 \\ y_i' &= m_3 x_i + m_4 y_i + t_2 \end{aligned}$$



$$\begin{pmatrix} x_i' \\ y_i' \end{pmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{pmatrix} x_i \\ y_i \end{pmatrix} + \begin{pmatrix} t_1 \\ t_2 \end{pmatrix}$$

$\boldsymbol{x_i'}$ $\quad$ $\boldsymbol{M}$ $\quad$ $\boldsymbol{x_i}$ $\quad$ $\boldsymbol{t}$

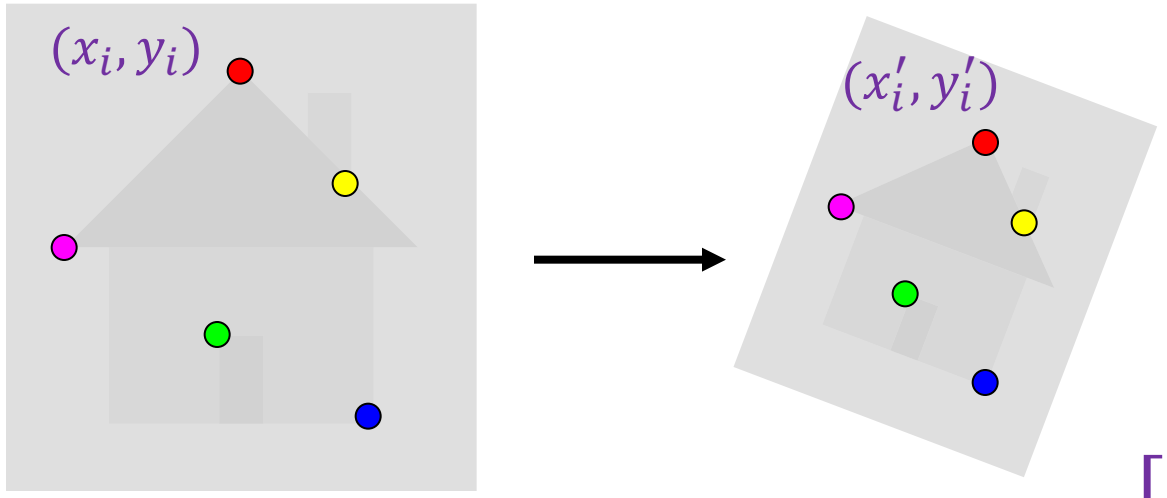
x_i' $\quad$ M $\quad$ x_i $\quad + \; T$ ← for a data point

Want to find $\boldsymbol{M}$, $\boldsymbol{t}$ to minimize

$$\sum_{i=1}^{n} \|\boldsymbol{x_i'} - \boldsymbol{M}\boldsymbol{x_i} - \boldsymbol{t}\|^2$$

Fitting an affine transformation

- Assume we know the correspondences, how do we get the transformation?



$$x_i' = m_1 x_i + m_2 y_i + t_1$$

$$\begin{pmatrix} x_i' \\ y_i' \end{pmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{pmatrix} x_i \\ y_i \end{pmatrix} + \begin{pmatrix} t_1 \\ t_2 \end{pmatrix}$$

one point match gives two equations

$$\begin{bmatrix} x_i & y_i & 0 & 0 & 1 & 0 \\ & & & & & \end{bmatrix} \begin{pmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{pmatrix} = \begin{pmatrix} \cdots \\ x_i' \\ y_i' \\ \cdots \end{pmatrix}$$

Summary

- 4 transformation types
  1. Isometries ⇒ 3 dof
  2. Similarity ⇒ 4 dof
  3. affine ⇒ 6 dof
  4. projective ⇒ 8 dof

for homogeneous $kx = H_p x$

homogeneous decomposition $H = H_S H_A H_P$

$$= \begin{bmatrix} sR & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} K & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} I & 0 \\ v^T & v \end{bmatrix}$$

Invariant properties ⇒ concurrency / collinearity / cross ratio / tangency.

In affine we can write $\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{bmatrix} a_1 & a_2 \\ a_3 & a_4 \end{bmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}$

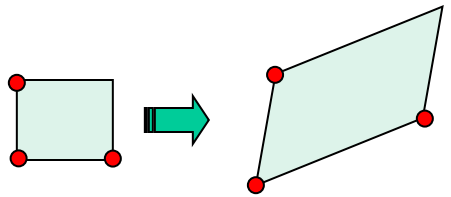
$$x' = Mx + t$$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} & & & & & \end{pmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \\ t_x \\ t_y \end{bmatrix}$$

1 point = 2 equations

Fitting an affine transformation

- How many matches do we need to solve for the transformation parameters?



need 3 point matches

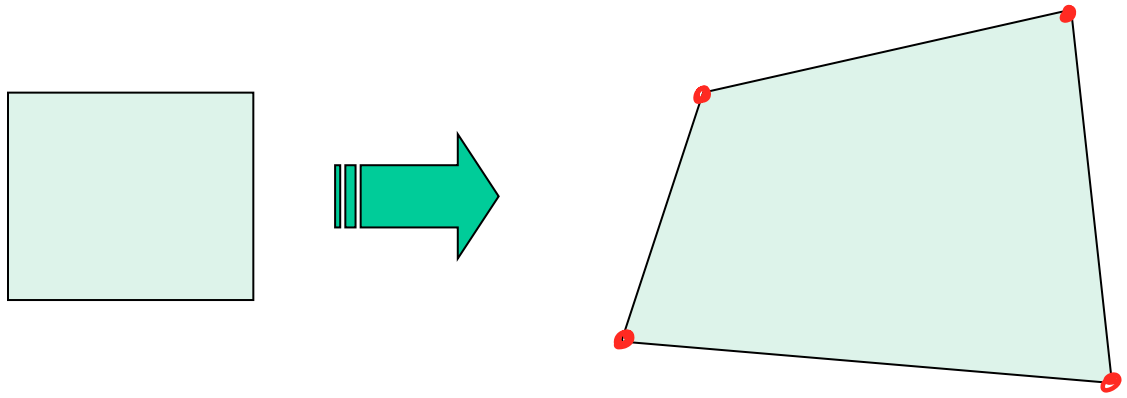
need to find 6 parameters

1 match gives 2 equations $\Rightarrow$

$$\begin{bmatrix} \cdots & & & & & \\ x_i & y_i & 0 & 0 & 1 & 0 \\ 0 & 0 & x_i & y_i & 0 & 1 \\ & & \cdots & & & \end{bmatrix} \begin{pmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{pmatrix} = \begin{pmatrix} \cdots \\ x_i' \\ y_i' \\ \cdots \end{pmatrix}$$

Fitting a plane projective transformation

- **Homography:** plane projective transformation (transformation taking a quad to another arbitrary quad)



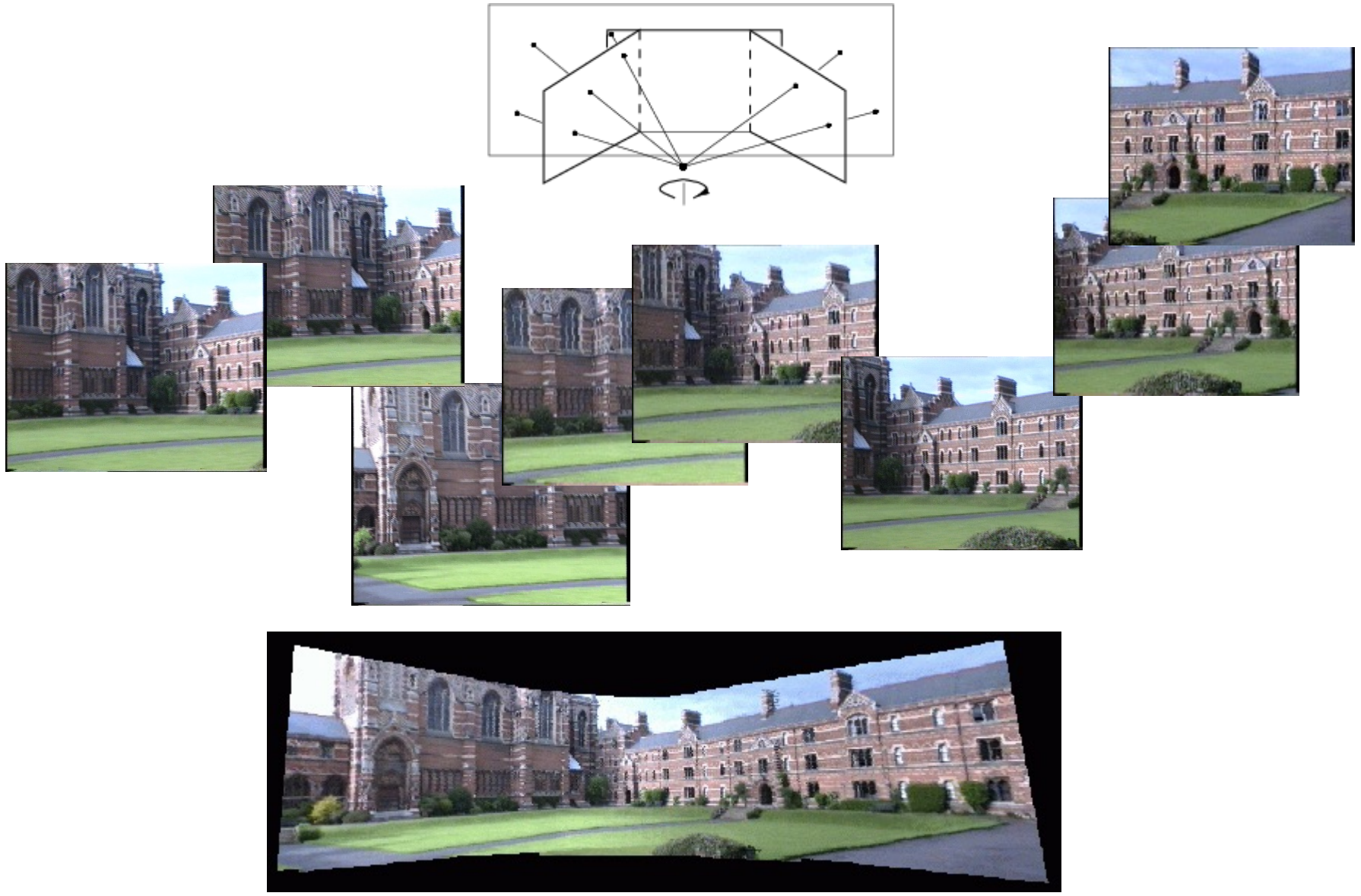
need 4 points to fit homography

Where Do Homographies Arise in the Real World?

The transformation between two views of a planar surface

The transformation between images from two cameras that share the same center

Application: Panorama Stitching



Source: Hartley & Zisserman

Fitting a Homography

2D homogeneous coordinates

Remember this

$$(x, y) \Rightarrow \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Converting *to* homogeneous image coordinates

$$\begin{pmatrix} x \\ y \\ w \end{pmatrix} \Rightarrow (x/w, y/w)$$

Converting *from* homogeneous image coordinates

Equation for homography in homogeneous coordinates:

$$\lambda \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$\lambda \boldsymbol{x}' = \boldsymbol{H}\boldsymbol{x}$$

homography transformation

Fitting a Homography

if $\underset{\sim}{a} = \underset{\sim}{b}$ then $\underset{\sim}{a} \times \underset{\sim}{b} = \underset{\sim}{0}$

Constraint from a match $(\boldsymbol{x}_i, \boldsymbol{x}_i')$: $\lambda \boldsymbol{x}_i' = \boldsymbol{H}\boldsymbol{x}_i$

How can we get rid of the scale factor λ?

Cross product trick: $\boldsymbol{x}_i' \times \boldsymbol{H}\boldsymbol{x}_i = \boldsymbol{0}$ ← cross product with itself is zero

Recall cross product:

$$\begin{pmatrix} a \\ b \\ c \end{pmatrix} \times \begin{pmatrix} a' \\ b' \\ c' \end{pmatrix} = \begin{pmatrix} bc' - b'c \\ ca' - c'a \\ ab' - a'b \end{pmatrix}$$ → from determinant

Let $\boldsymbol{h}_1^T$, $\boldsymbol{h}_2^T$, $\boldsymbol{h}_3^T$ be the rows of $\boldsymbol{H}$. Then

$$\boldsymbol{x}_i' \times \boldsymbol{H}\boldsymbol{x}_i = \begin{pmatrix} x_i' \\ y_i' \\ 1 \end{pmatrix} \times \begin{pmatrix} \boldsymbol{h}_1^T \boldsymbol{x}_i \\ \boldsymbol{h}_2^T \boldsymbol{x}_i \\ \boldsymbol{h}_3^T \boldsymbol{x}_i \end{pmatrix} = \begin{pmatrix} y_i' \boldsymbol{h}_3^T \boldsymbol{x}_i - \boldsymbol{h}_2^T \boldsymbol{x}_i \\ \boldsymbol{h}_1^T \boldsymbol{x}_i - x_i' \boldsymbol{h}_3^T \boldsymbol{x}_i \\ x_i' \boldsymbol{h}_2^T \boldsymbol{x}_i - y_i' \boldsymbol{h}_1^T \boldsymbol{x}_i \end{pmatrix}$$

inner product of row of H and xi column vector

single value

row

col

Fitting a Homography

Constraint from a match $(\boldsymbol{x}_i, \boldsymbol{x}_i')$:

$$\boldsymbol{x}_i' \times \boldsymbol{H}\boldsymbol{x}_i = \begin{pmatrix} x_i' \\ y_i' \\ 1 \end{pmatrix} \times \begin{pmatrix} \boldsymbol{h}_1^T \boldsymbol{x}_i \\ \boldsymbol{h}_2^T \boldsymbol{x}_i \\ \boldsymbol{h}_3^T \boldsymbol{x}_i \end{pmatrix} = \begin{pmatrix} y_i' \boldsymbol{h}_3^T \boldsymbol{x}_i - \boldsymbol{h}_2^T \boldsymbol{x}_i \\ \boldsymbol{h}_1^T \boldsymbol{x}_i - x_i' \boldsymbol{h}_3^T \boldsymbol{x}_i \\ x_i' \boldsymbol{h}_2^T \boldsymbol{x}_i - y_i' \boldsymbol{h}_1^T \boldsymbol{x}_i \end{pmatrix} = \textcircled{c}$$

Rearranging the terms:

we need to find these!

$$\begin{bmatrix} \textcircled{1}\ \boldsymbol{0}^T & -\boldsymbol{x}_i^T & y_i' \boldsymbol{x}_i^T \\ \textcircled{2}\ \boldsymbol{x}_i^T & \boldsymbol{0}^T & -x_i' \boldsymbol{x}_i^T \\ \textcircled{3}\ -y_i' \boldsymbol{x}_i^T & x_i' \boldsymbol{x}_i^T & \boldsymbol{0}^T \end{bmatrix} \begin{pmatrix} \boldsymbol{h}_1 \\ \boldsymbol{h}_2 \\ \boldsymbol{h}_3 \end{pmatrix} = 0$$

$-\boldsymbol{x}_i^T \boldsymbol{h}_1 = -\boldsymbol{h}_1^T \boldsymbol{x}_i$ since both are [ ][ ] like dot product of vectors.

Are these equations independent?

No ③ is a linear combination of ① and ②

In homogeneous $\lambda x' = Hx$ so

$$x' \times Hx = 0$$

Let rows of H be $h_1^T \; h_2^T \; h_3^T$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} \times \begin{bmatrix} h_1^T x \\ h_2^T x \\ h_3^T x \end{bmatrix} = 0$$

$$\begin{bmatrix} 0 & -x^T & x^T y' \\ x^T & 0 & -x^T x' \\ -x^T y' & x^T x' & 0 \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix}$$

$$A \qquad h = 0$$

$A^T A$ smallest eigen value

$\Downarrow$

eigen vector

$\Downarrow$

h

Fitting a Homography

we need to solve this for h's

Final linear system:

for n points

$$\begin{bmatrix} \mathbf{0}^T & -\boldsymbol{x}_1^T & +y_1'\boldsymbol{x}_1^T \\ \boldsymbol{x}_1^T & \mathbf{0}^T & -x_1'\boldsymbol{x}_1^T \\ \dots & \dots & \dots \\ \mathbf{0}^T & -\boldsymbol{x}_n^T & +y_n'\boldsymbol{x}_n^T \\ \boldsymbol{x}_n^T & \mathbf{0}^T & -x_n'\boldsymbol{x}_n^T \end{bmatrix} \begin{pmatrix} \boldsymbol{h}_1 \\ \boldsymbol{h}_2 \\ \boldsymbol{h}_3 \end{pmatrix} = 0 \qquad A\boldsymbol{h} = 0$$

make sure the norm is equal to 1

each point match gives 2 equations

Homogeneous least squares: find $\boldsymbol{h}$ minimizing $\|A\boldsymbol{h}\|^2$

Solution is eigenvector of A^TA corresponding to smallest eigenvalue

What is the minimum number of matches needed for a solution?

need 4 point matches

2. Four: A has 8 degrees of freedom (9 parameters, but scale is arbitrary), one match gives us two linearly independent equations

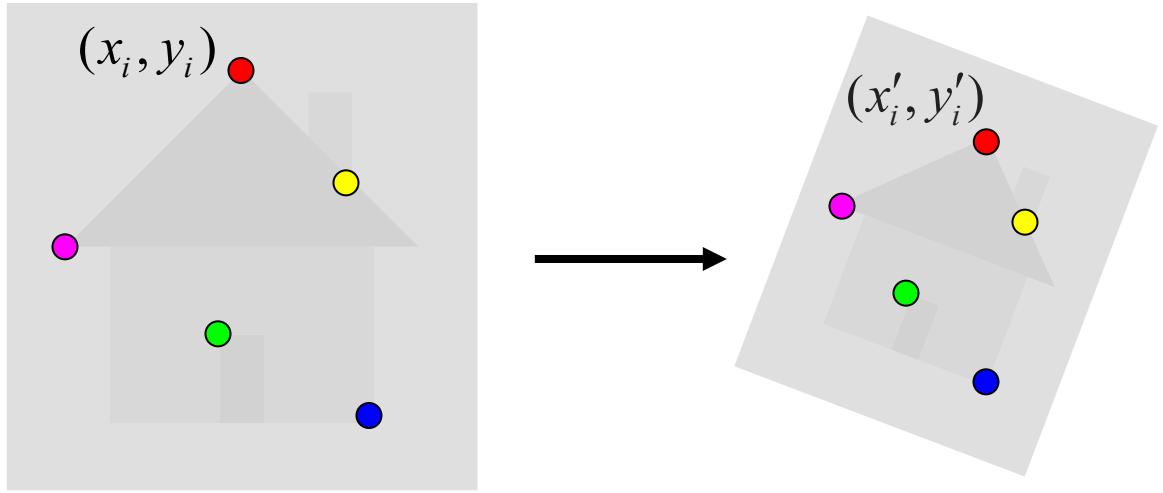
Alignment: Overview

- Motivation
- Fitting of transformations
  - Affine transformations
  - Homographies
- Robust alignment
  - Descriptor-based feature matching
  - RANSAC

Robust Feature-Based Alignment

So far, we've assumed that we are given a set of correspondences between the two images we want to align

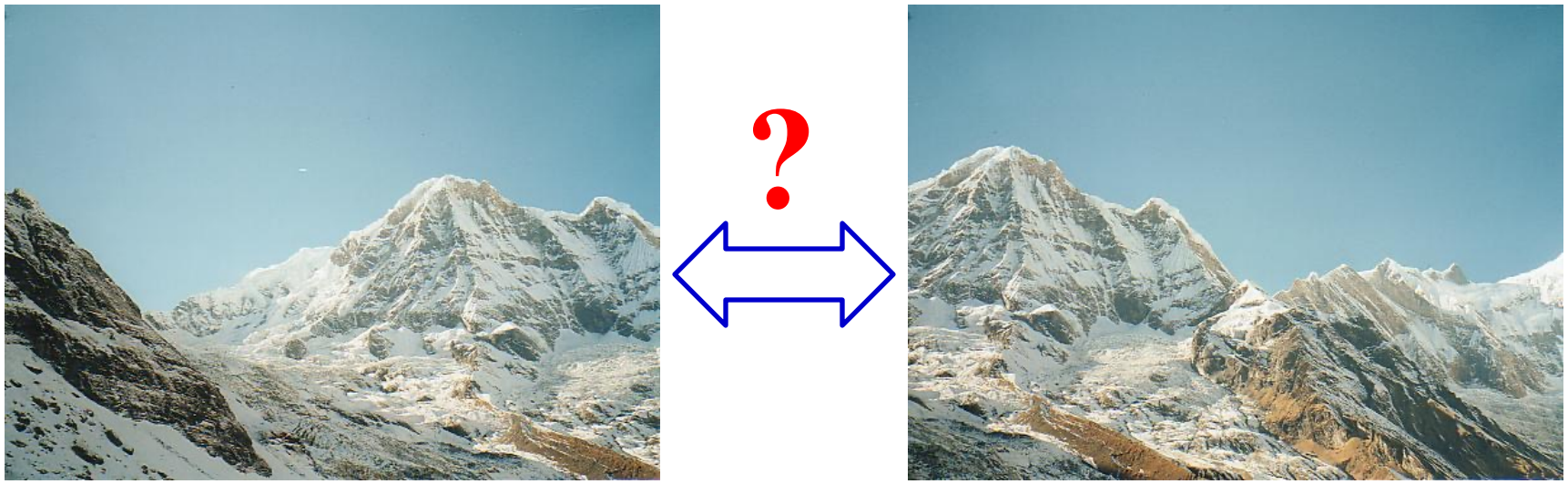
What if we don't know the correspondences?



Robust Feature-Based Alignment

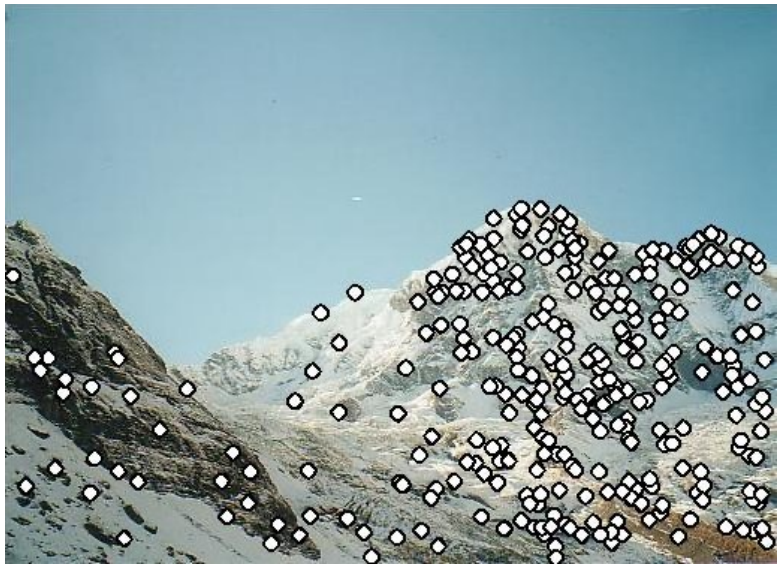
So far, we've assumed that we are given a set of correspondences between the two images we want to align

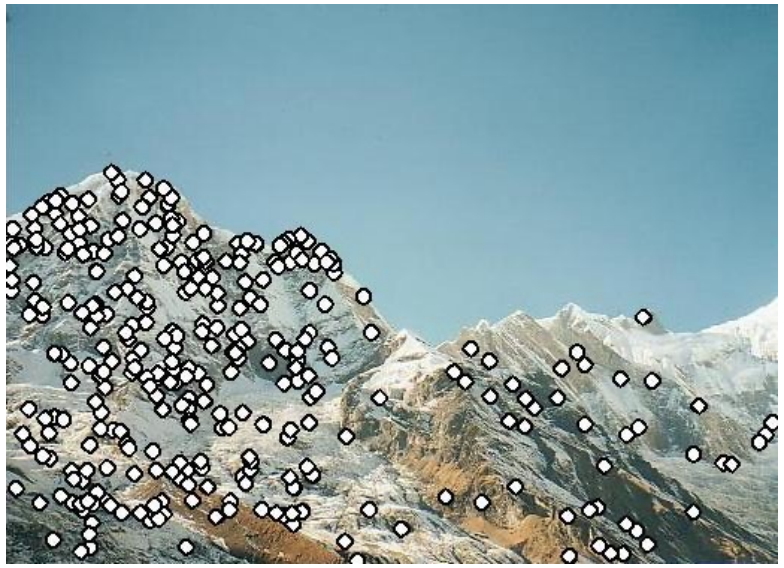
What if we don't know the correspondences?



Robust Feature-Based Alignment

Robust Feature-Based Alignment



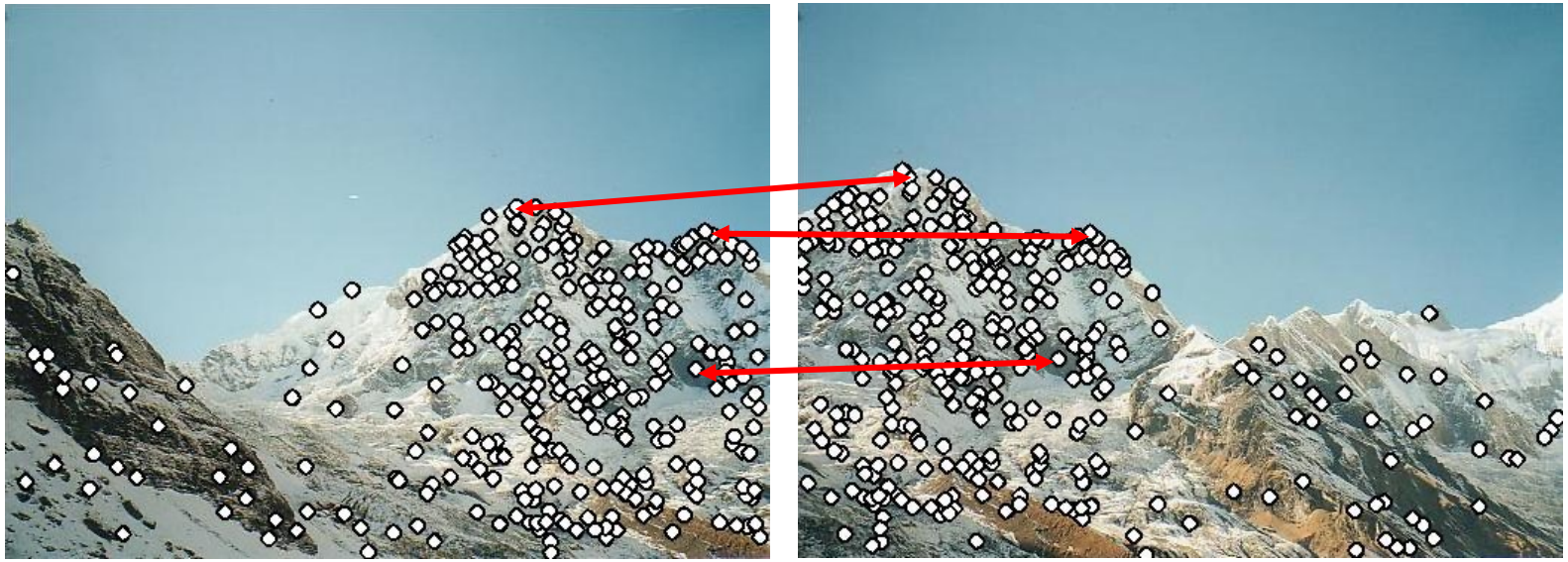


- Extract features

Robust Feature-Based Alignment

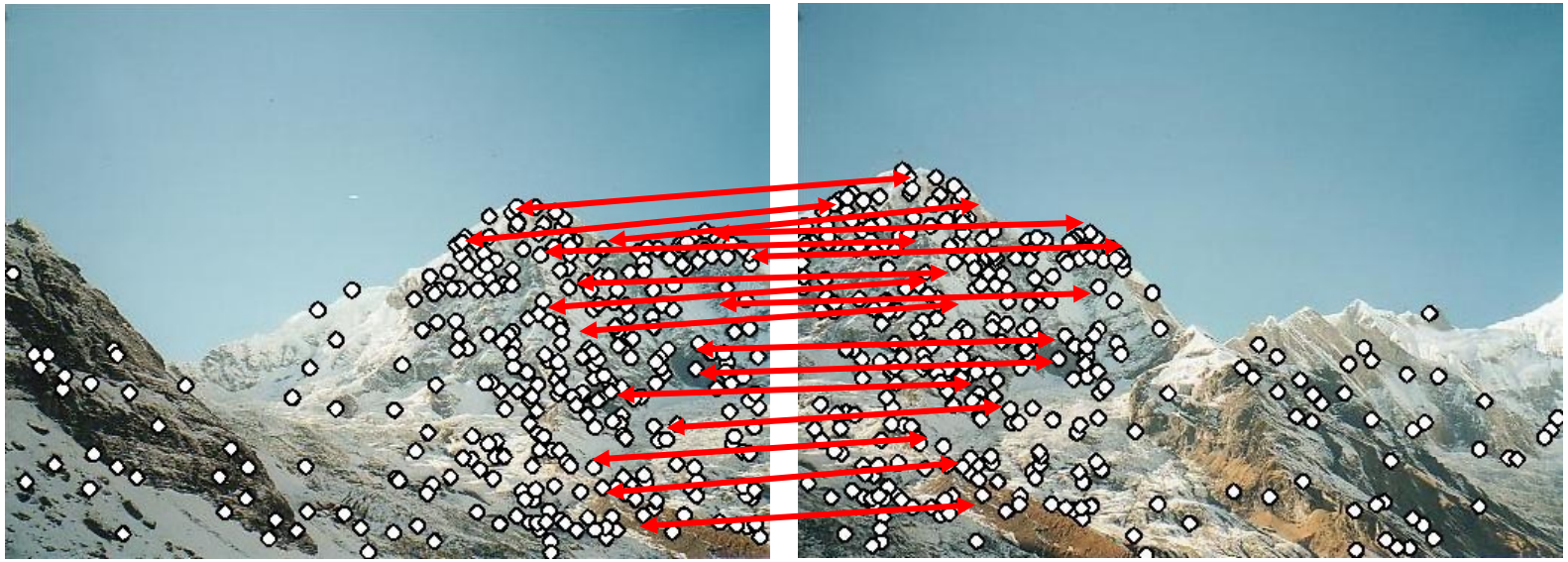
- Extract features
- Compute *putative matches*

Robust Feature-Based Alignment



- Extract features
- Compute *putative matches*
- Loop:
  - *Hypothesize* transformation T

Robust Feature-Based Alignment

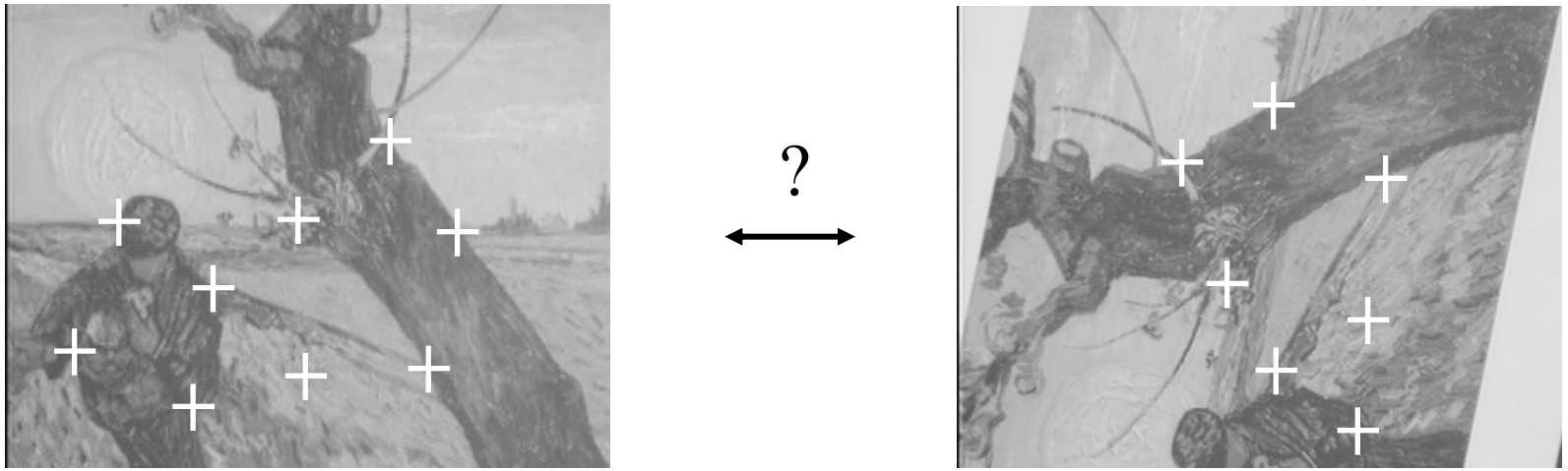


- Extract features
- Compute *putative matches*
- Loop:
  - *Hypothesize* transformation T
  - *Verify* transformation (search for other matches consistent with T)

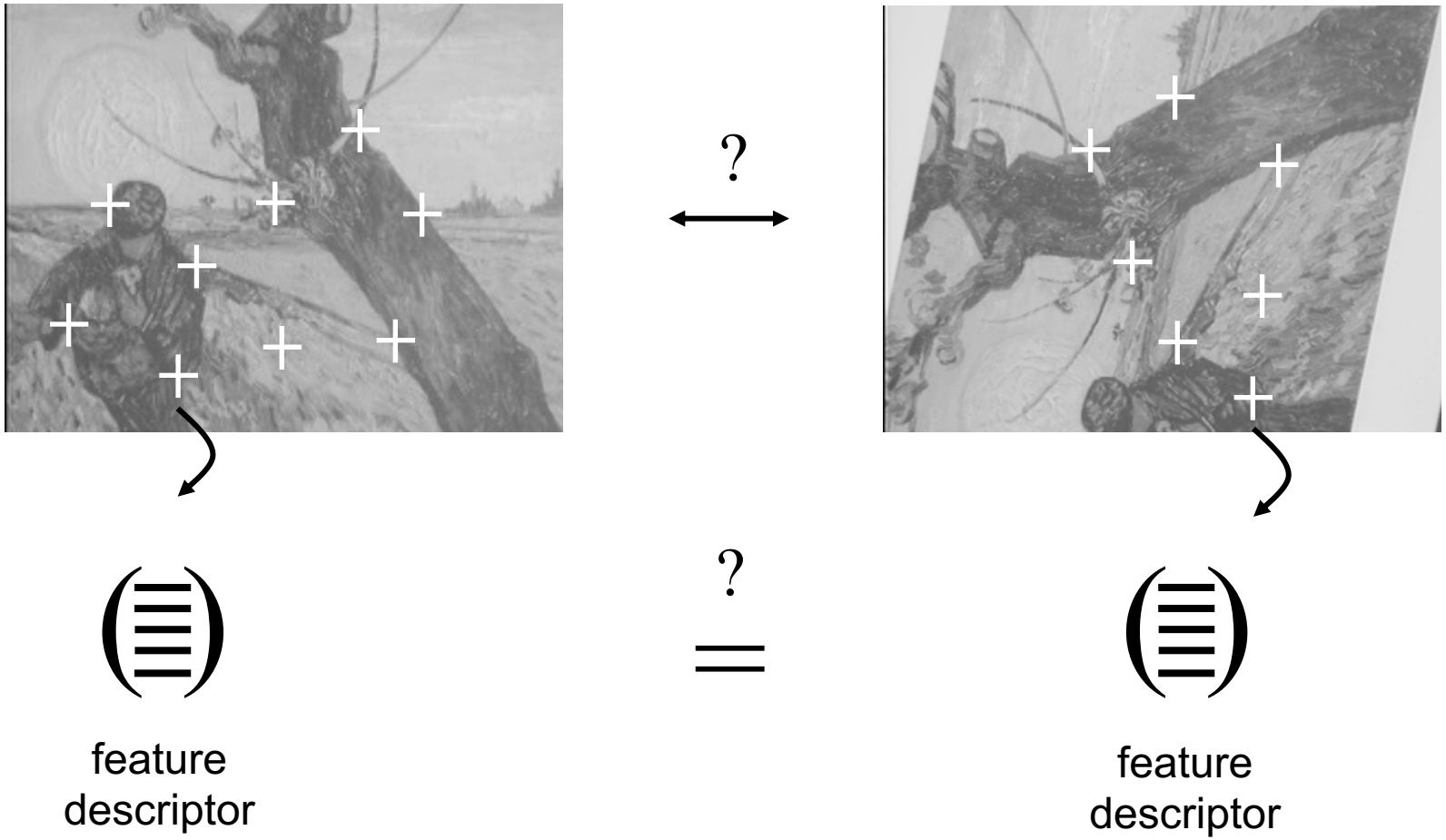
Robust Feature-Based Alignment

- Extract features
- Compute *putative matches*
- Loop:
  - *Hypothesize* transformation T
  - *Verify* transformation (search for other matches consistent with T)

Generating Putative Correspondences



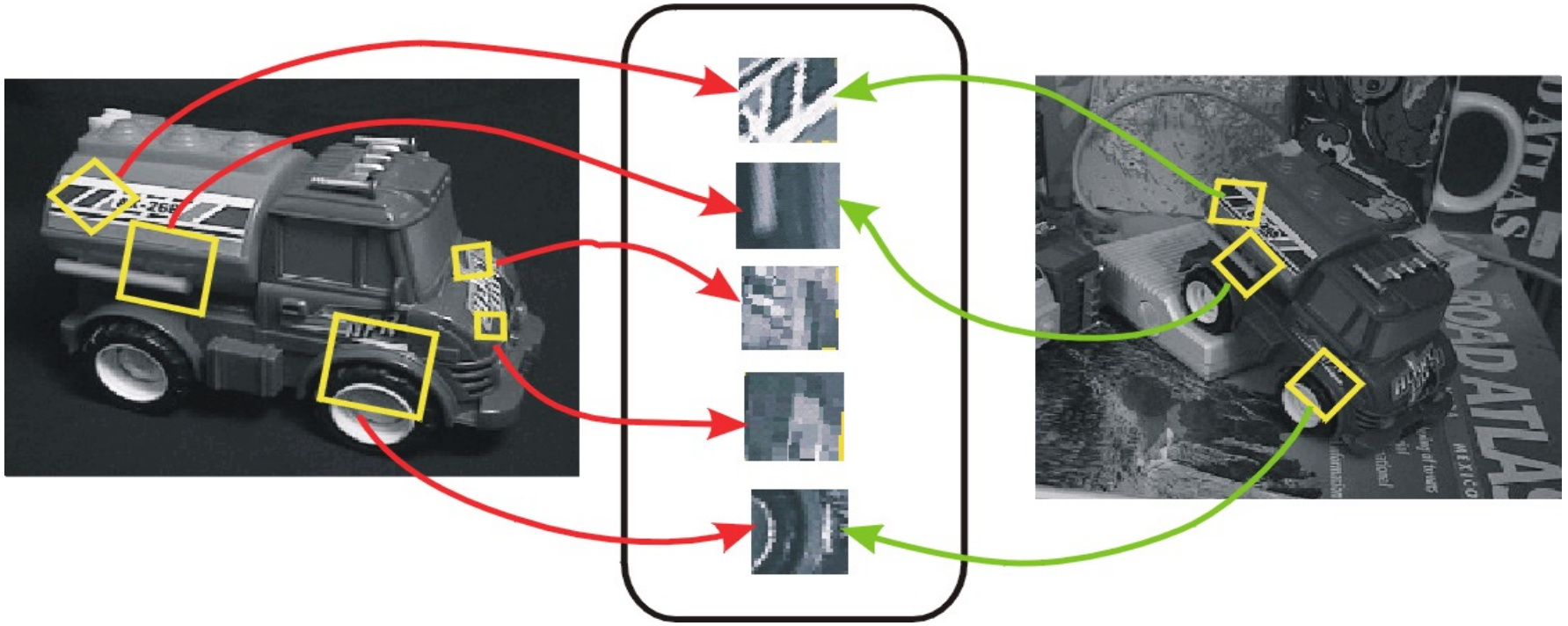
Generating Putative Correspondences



- Need to compare *feature descriptors* of local patches surrounding interest points

Feature Descriptors

- Recall: feature detection vs. feature description



Comparing Feature Descriptors

- Simplest descriptor: vector of raw intensity values
- How to compare two such vectors $\boldsymbol{u}$ and $\boldsymbol{v}$?
  - **Sum of squared differences** (SSD):

$$\mathrm{SSD}(\boldsymbol{u}, \boldsymbol{v}) = \sum_i (u_i - v_i)^2$$

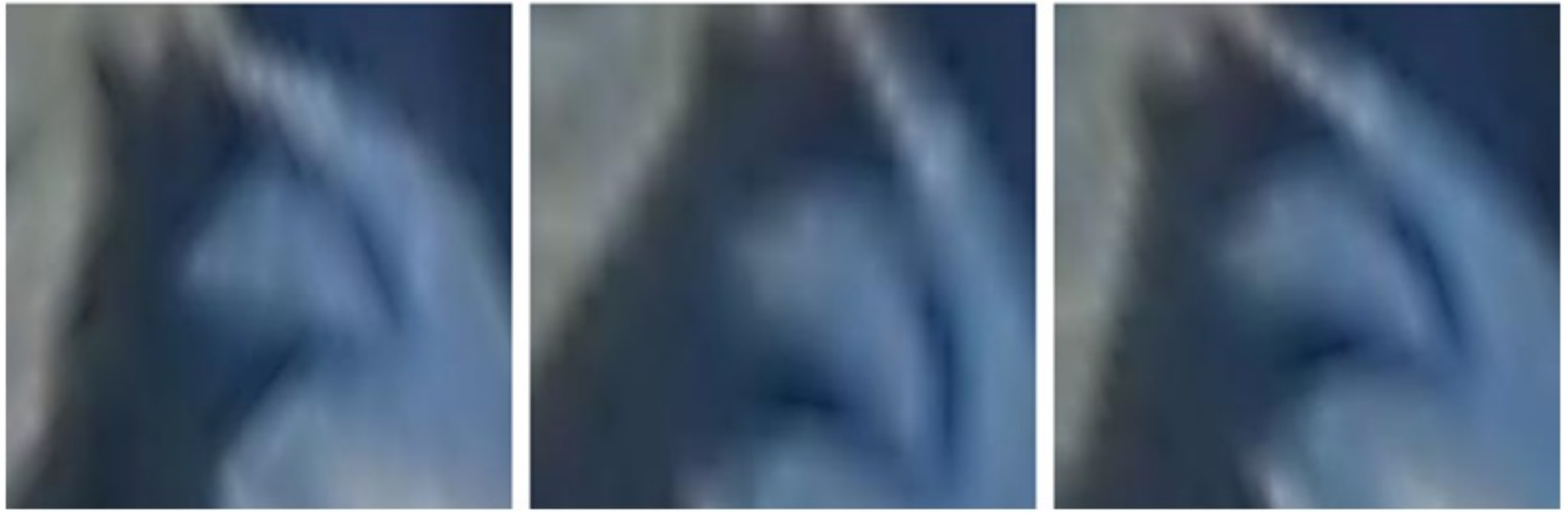
  - **Normalized correlation**: dot product between $\boldsymbol{u}$ and $\boldsymbol{v}$ normalized to have zero mean and unit norm:

$$\rho(\boldsymbol{u}, \boldsymbol{v}) = \frac{\sum_i (u_i - \overline{\boldsymbol{u}})(v_i - \overline{\boldsymbol{v}})}{\sqrt{\left(\sum_j (u_j - \overline{\boldsymbol{u}})^2\right)\left(\sum_j (v_j - \overline{\boldsymbol{v}})^2\right)}}$$

  - Why would we prefer normalized correlation over SSD?

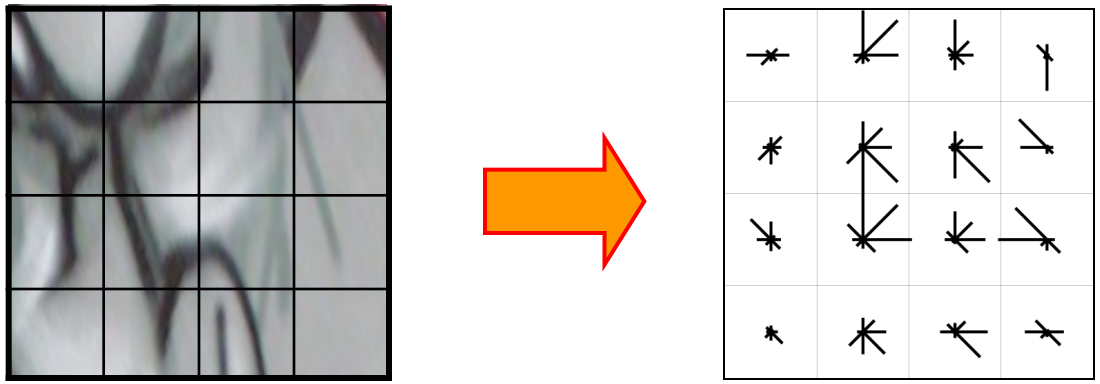
Disadvantage of Intensity Vectors as Descriptors

- Small deformations can affect the matching score a lot



Feature Descriptors: SIFT

- Descriptor computation:
  - Divide patch into 4×4 sub-patches
  - Compute histogram of gradient orientations (8 reference angles) inside each sub-patch
  - Resulting descriptor: $4\times4\times8 = 128$ dimensions



David G. Lowe. Distinctive image features from scale-invariant keypoints. *IJCV* 60 (2), pp. 91-110, 2004.

Feature Descriptors: SIFT

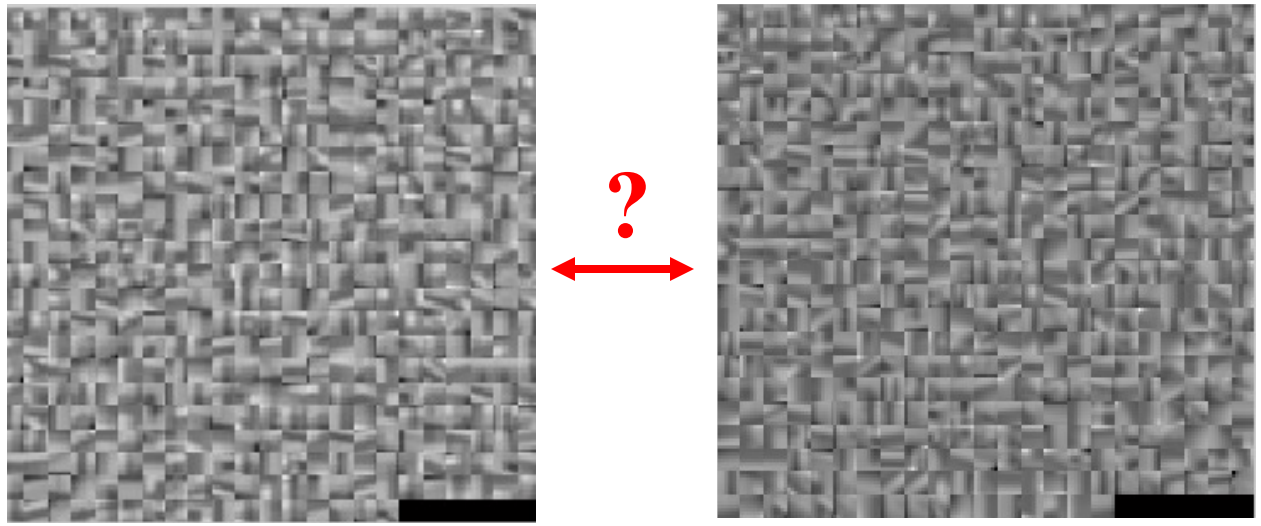
- Descriptor computation:
  - Divide patch into 4×4 sub-patches
  - Compute histogram of gradient orientations (8 reference angles) inside each sub-patch
  - Resulting descriptor: $4 \times 4 \times 8 = 128$ dimensions

- What are the advantages of SIFT descriptor over raw pixel values?
  - Gradients are less sensitive to illumination change
  - Pooling of gradients over the sub-patches achieves robustness to small shifts, but still preserves some spatial information

David G. Lowe. Distinctive image features from scale-invariant keypoints. *IJCV* 60 (2), pp. 91-110, 2004.

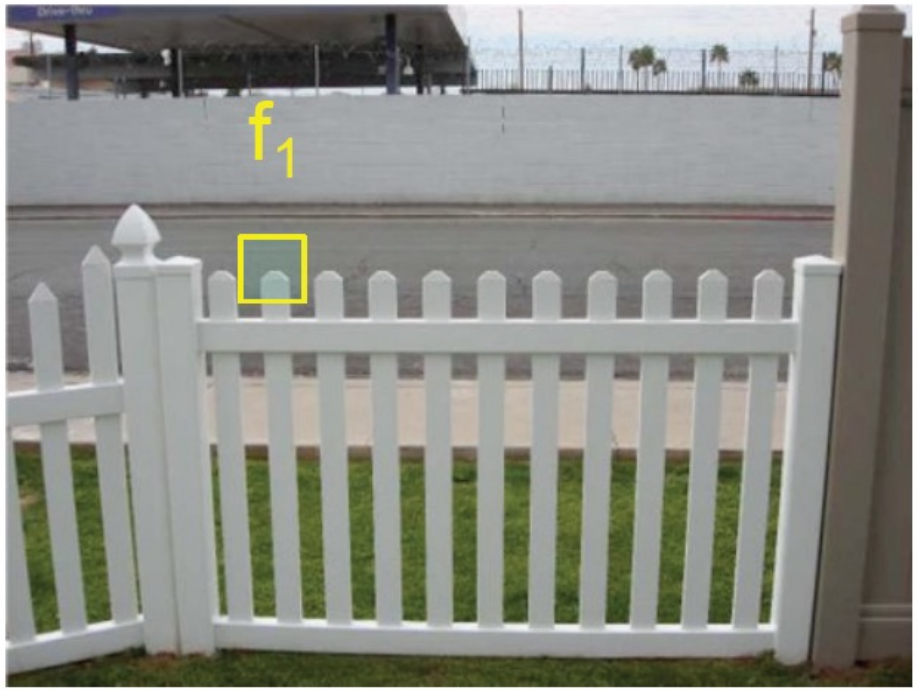
Generating Putative Correspondences

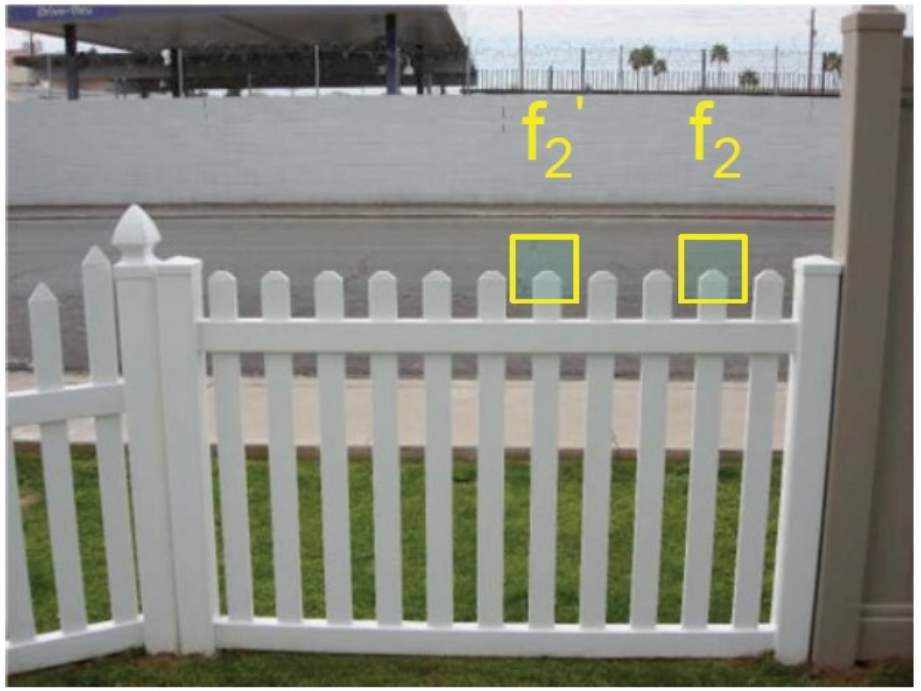
- For each patch in one image, find a short list of patches in the other image that could match it based solely on appearance



Rejection of Ambiguous Matches

- How can we tell which putative matches are more reliable?
- Heuristic: compare distance of nearest neighbor to that of second nearest neighbor

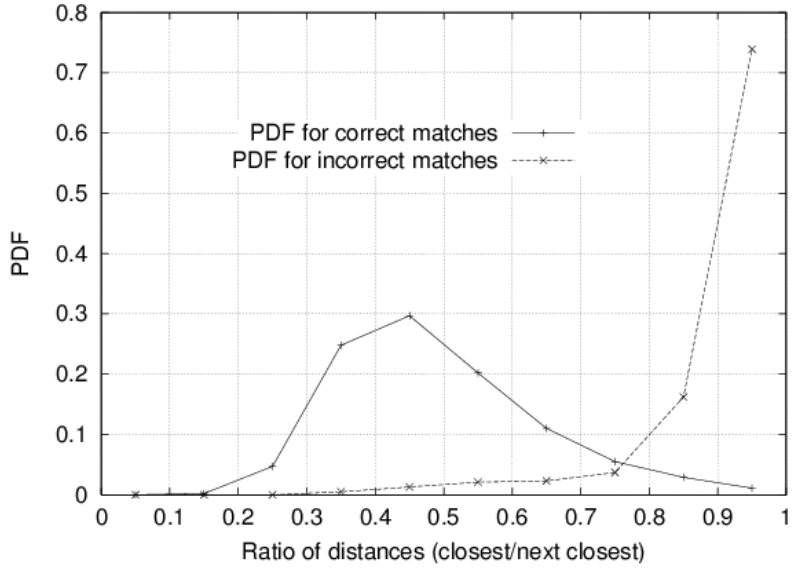




Source: Y. Furukawa

Rejection of Ambiguous Matches

- How can we tell which putative matches are more reliable?
- Heuristic: compare distance of **nearest** neighbor to that of **second nearest** neighbor
  - Ratio of closest distance to second-closest distance will be high for features that are not distinctive



Threshold of 0.8 found to provide good separation

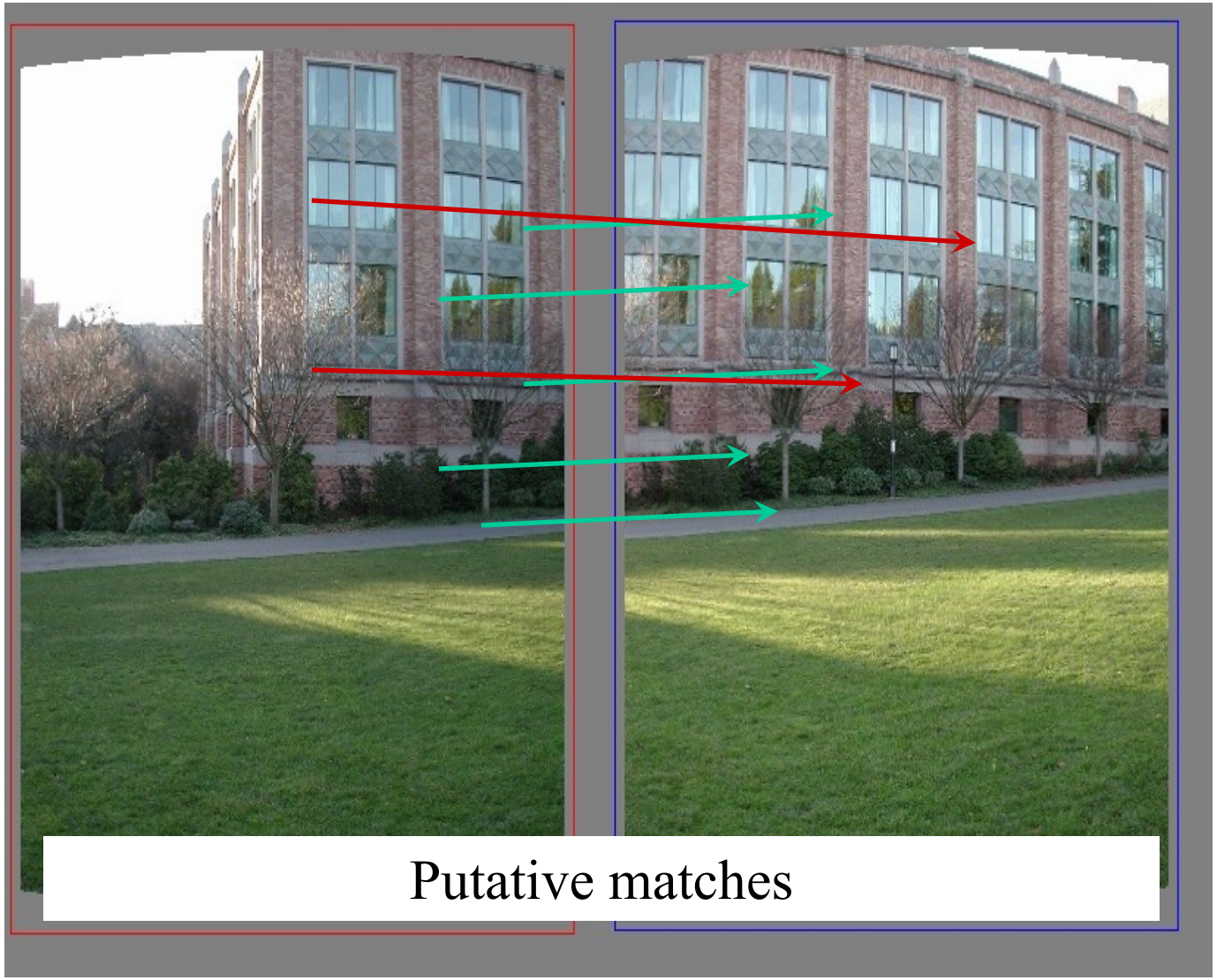
David G. Lowe. Distinctive image features from scale-invariant keypoints. *IJCV* 60 (2), pp. 91-110, 2004.

Robust Alignment

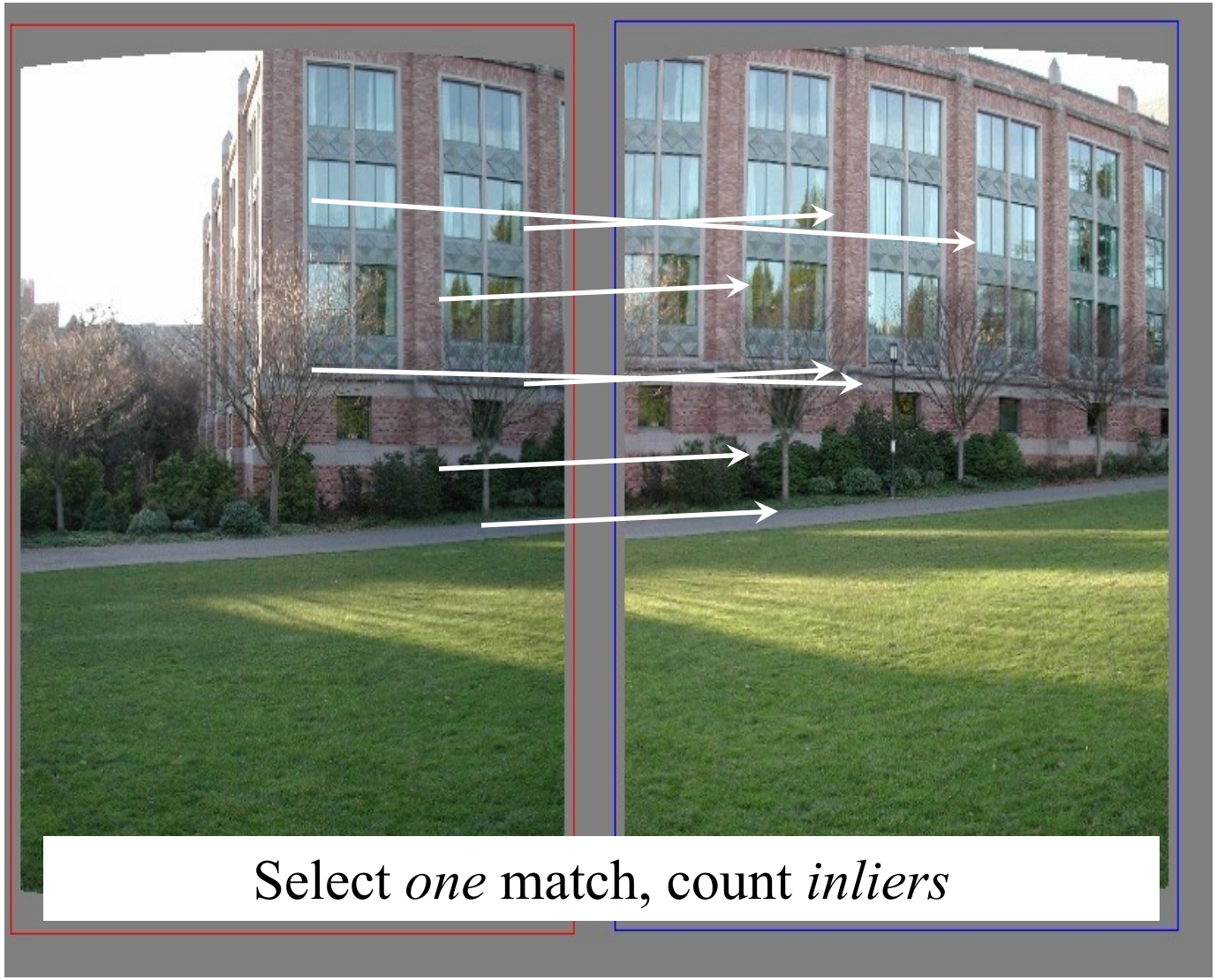
- Even after filtering out ambiguous matches, the set of putative matches still contains a very high percentage of outliers

- Solution: RANSAC
- RANSAC loop:
  1. Randomly select a *seed group* of matches
  2. Compute transformation from seed group
  3. Find *inliers* to this transformation
  4. If the number of inliers is sufficiently large, re-compute least-squares estimate of transformation on all of the inliers
- At the end, keep the transformation with the largest number of inliers

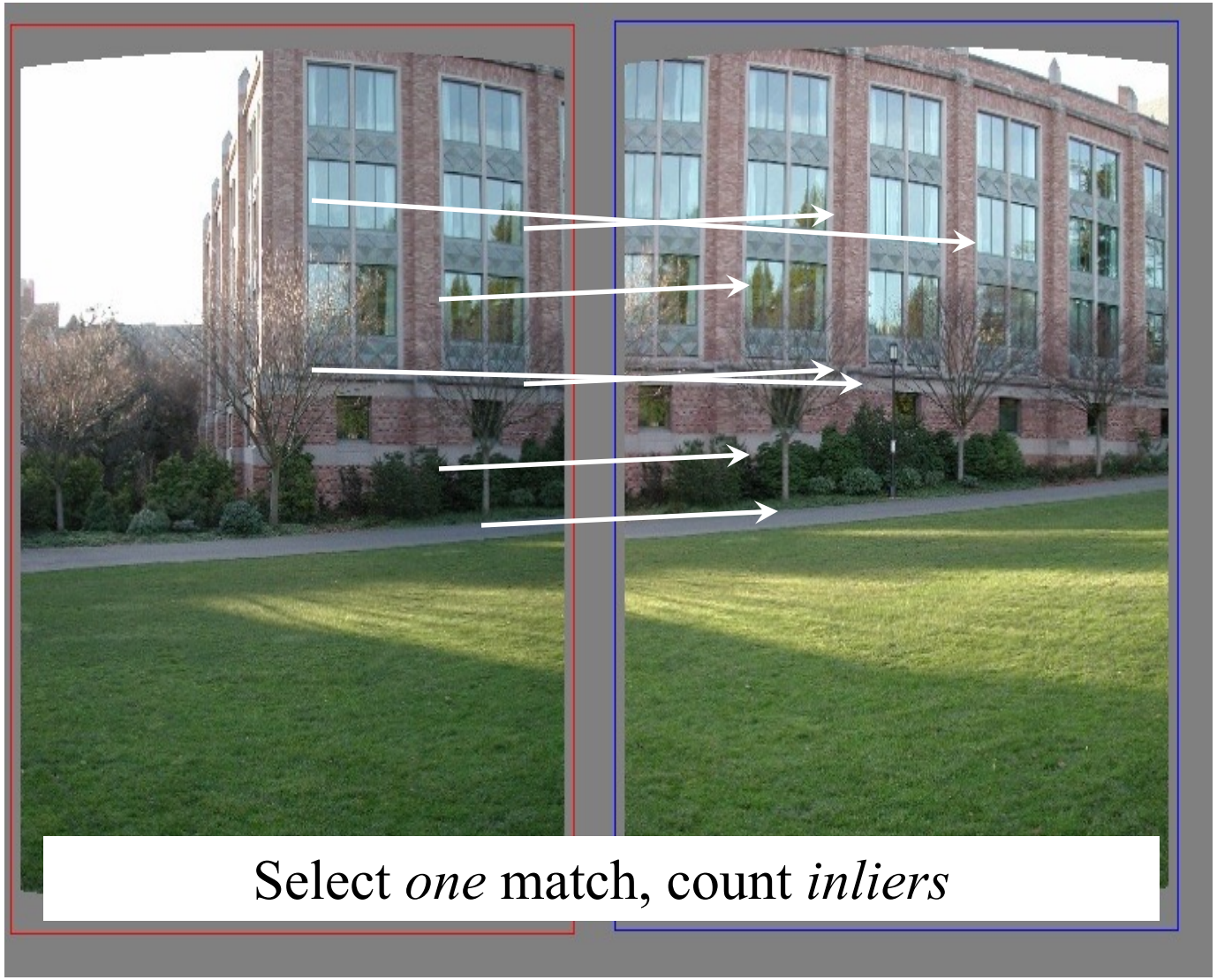
RANSAC Example: Translation



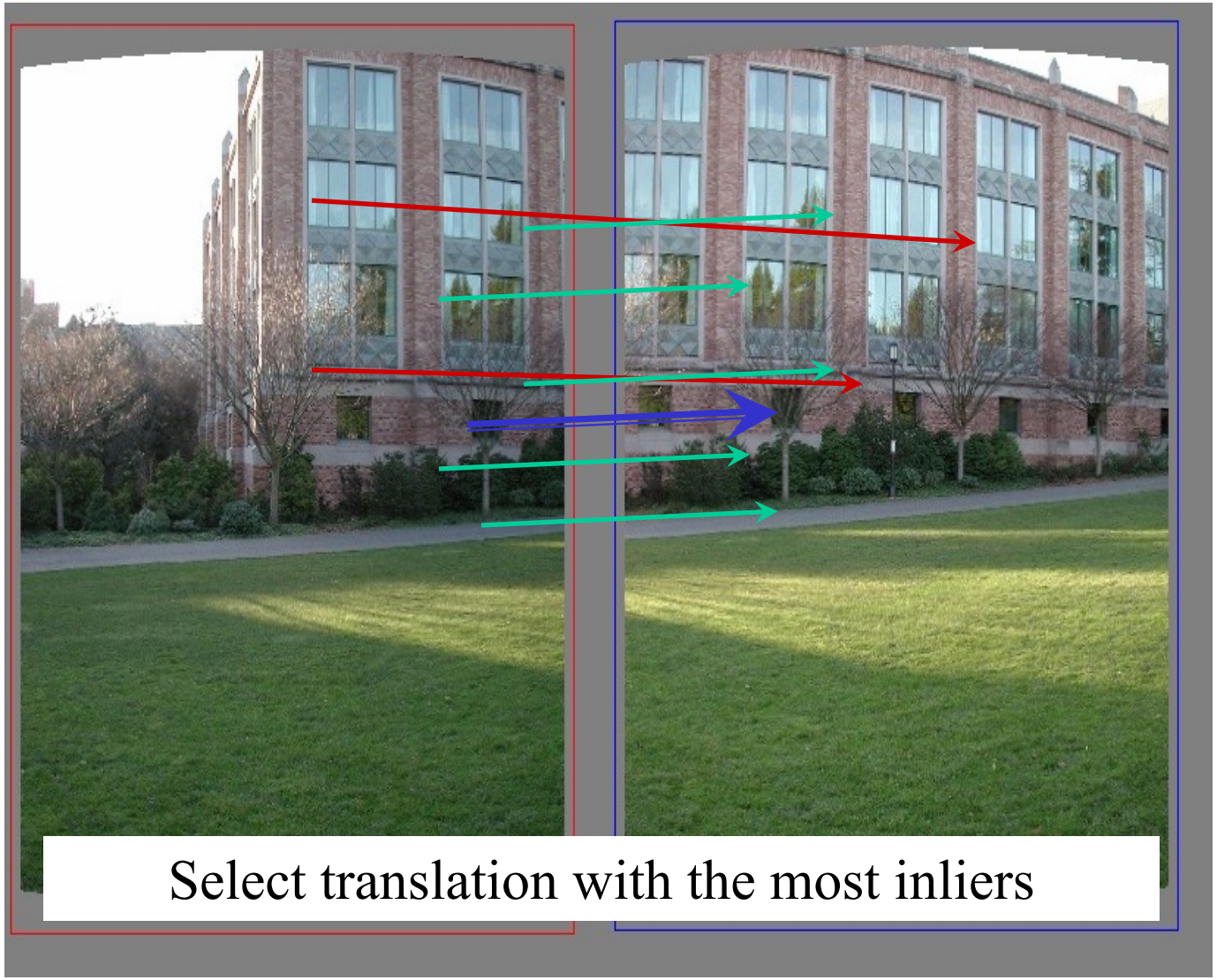
RANSAC Example: Translation



RANSAC Example: Translation

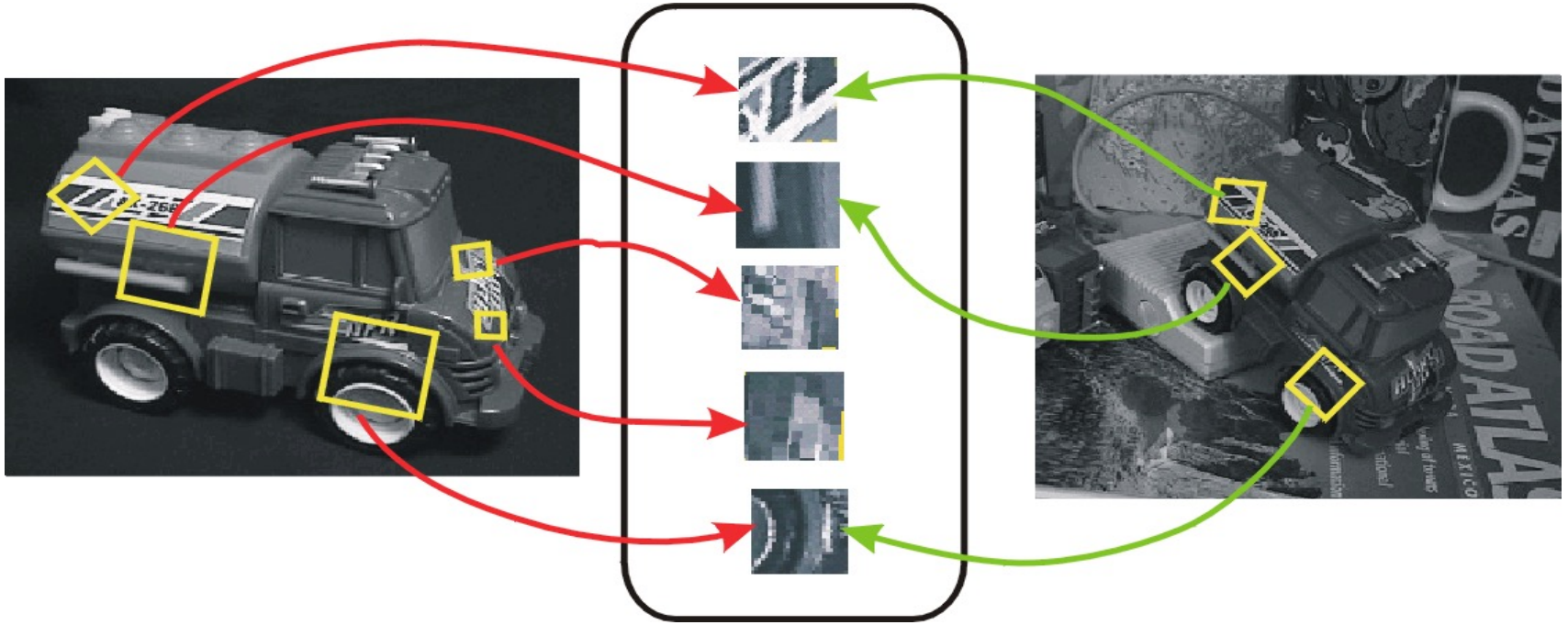


RANSAC example: Translation



Alternative for Robust Alignment: Hough Voting

- A single SIFT match can vote for translation, rotation, and scale parameters of a transformation between two images
  - Votes can be accumulated in a 4D Hough space with large bins
  - Clusters of matches falling into the same bin should undergo a more precise verification procedure



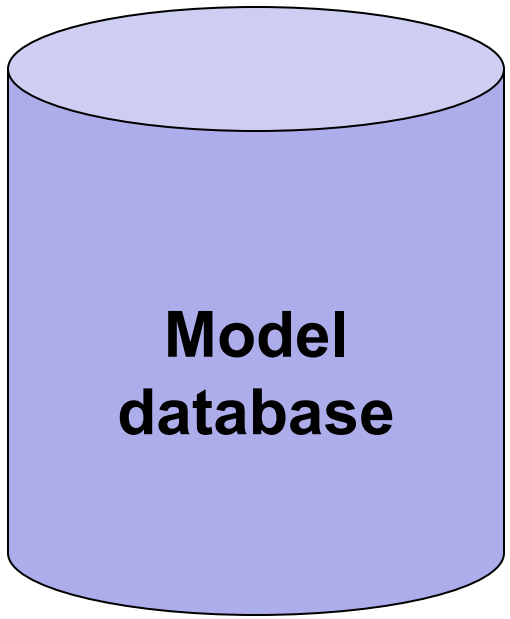
David G. Lowe. Distinctive image features from scale-invariant keypoints. *IJCV* 60 (2), pp. 91-110, 2004.

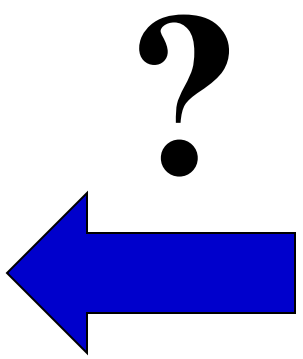
Alignment: Overview

- Motivation
- Fitting of transformations
  - Affine transformations
  - Homographies
- Robust alignment
  - Descriptor-based feature matching
  - RANSAC
- Large-scale alignment
  - Inverted indexing
  - Vocabulary trees

Scalability: Alignment to Large Databases

- What if we need to align a test image with thousands or millions of images in a model database?
  - Efficient putative match generation: approximate descriptor similarity search, inverted indices





Test image

Large-Scale Visual Search

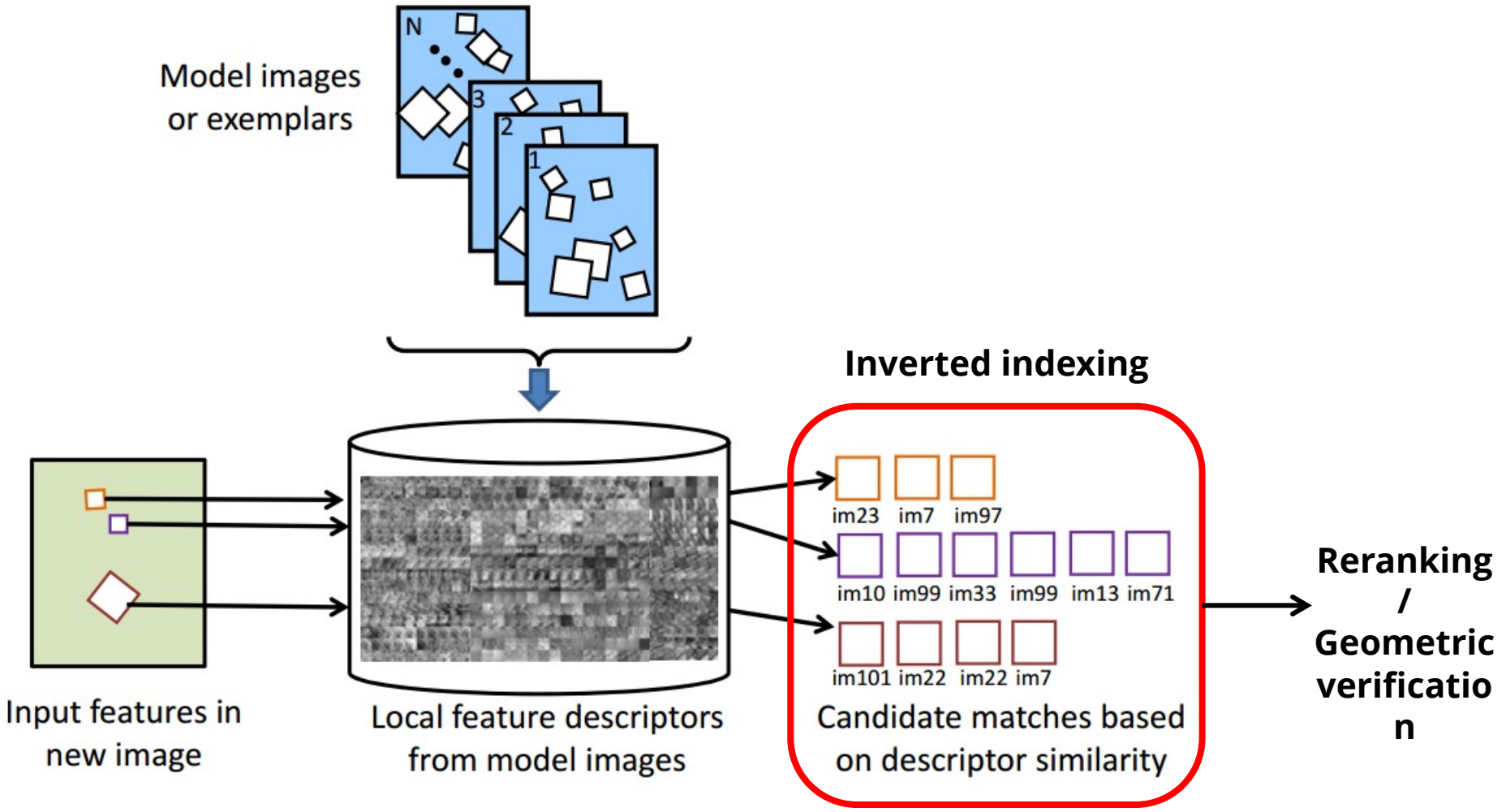
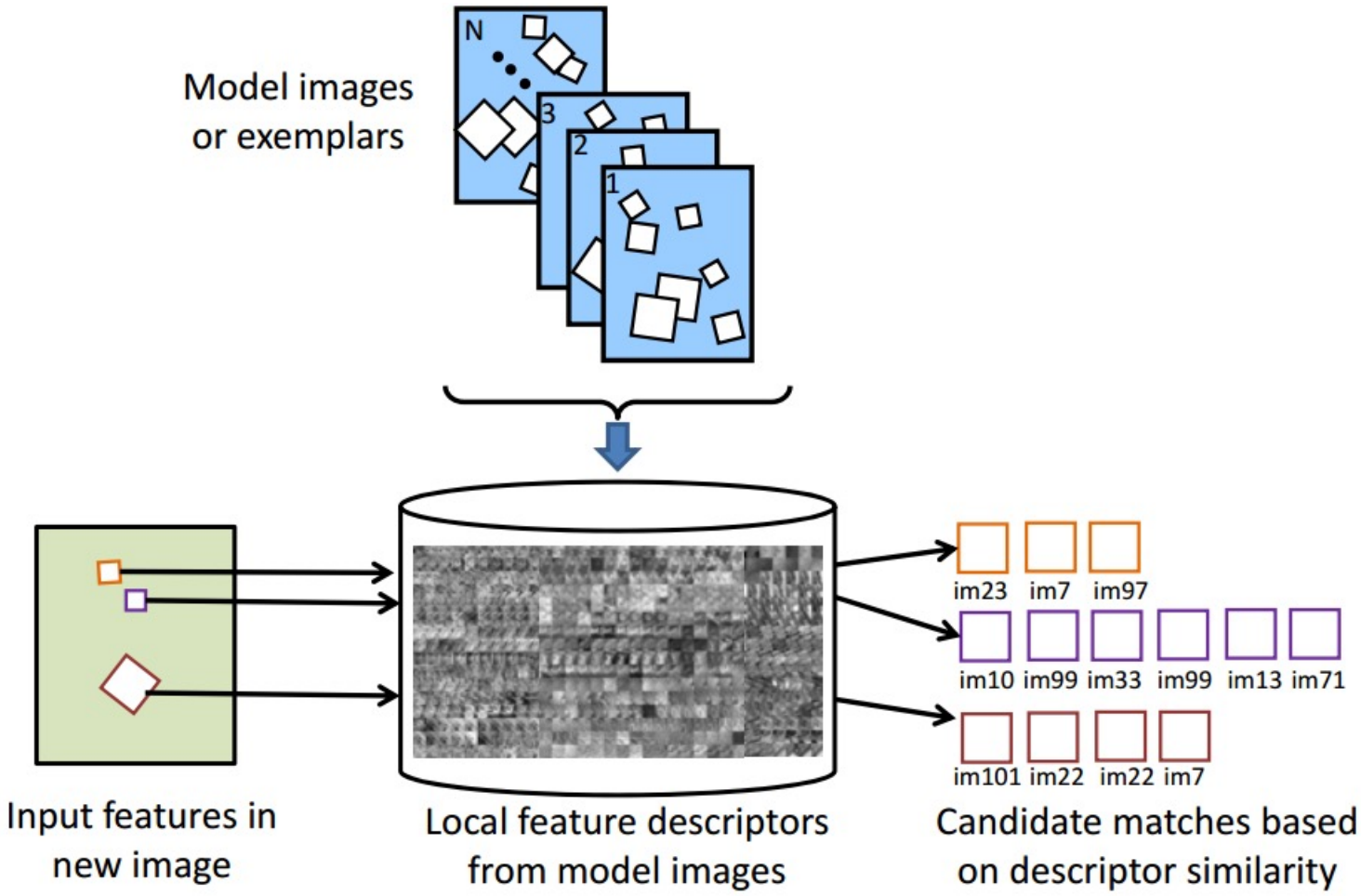


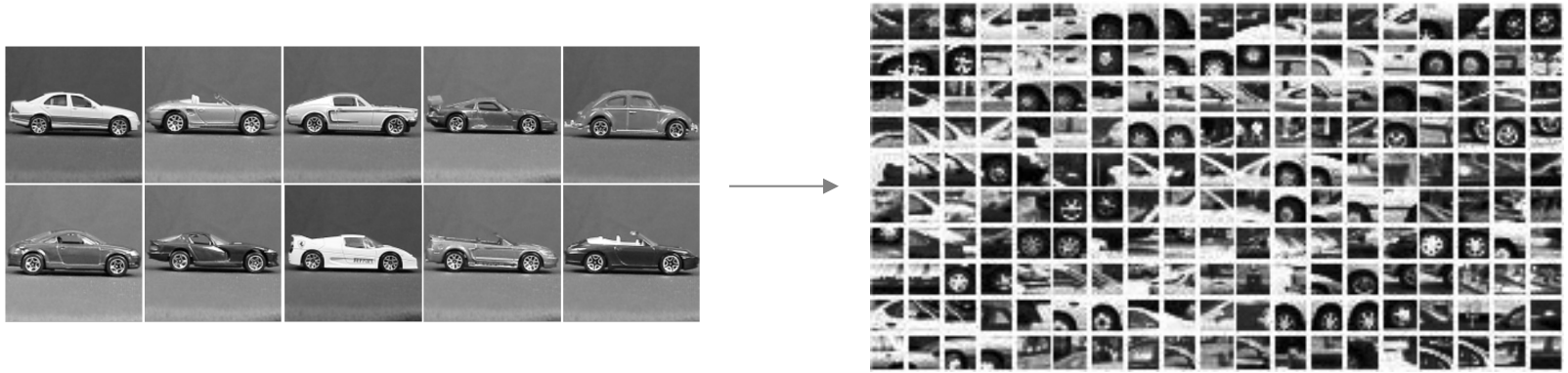
Figure from: Kristen Grauman and Bastian Leibe, Visual Object Recognition, Synthesis Lectures on Artificial Intelligence and Machine Learning, April 2011, Vol. 5, No. 2, Pages 1-181

How to Do the Indexing?

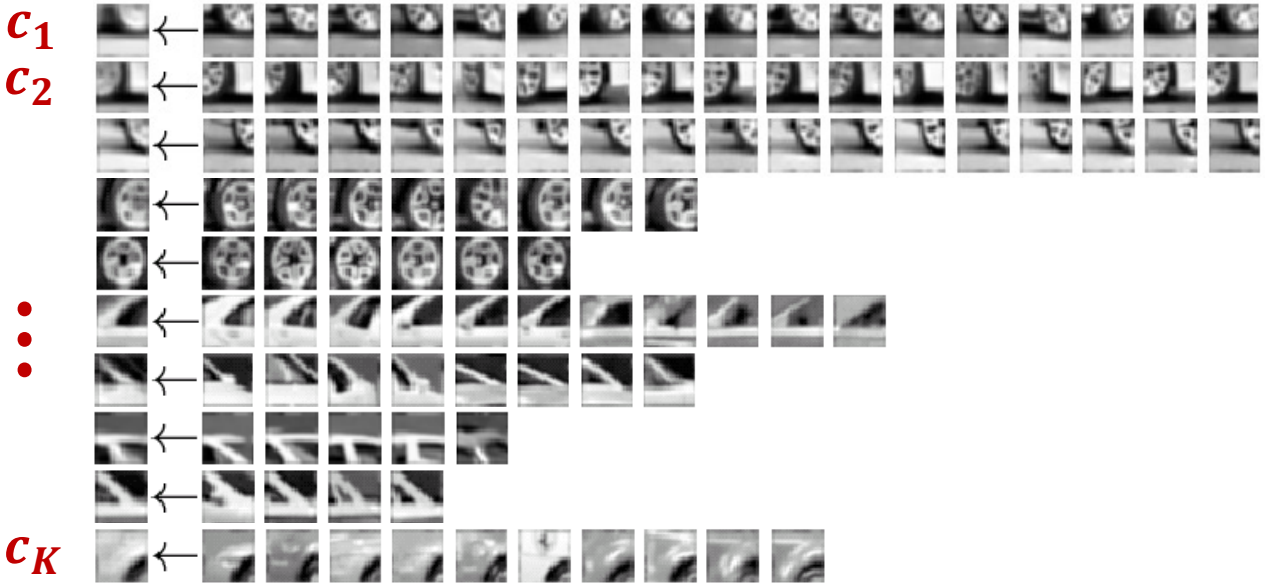


- Idea: find a set of *visual codewords* to which descriptors can be *quantized*

Recall: Visual Codebook for implicit shape models



Appearance codebook



B. Leibe, A. Leonardis, and B. Schiele, Combined Object Categorization and Segmentation with an Implicit Shape Model, ECCV Workshop on Statistical Learning in Computer Vision 2004

K-Means Clustering

- We want to find K cluster centers and an assignment of points to cluster centers to minimize the sum of squared Euclidean distances between each point and its assigned cluster center:

$$\sum_i \sum_k a_{ik} \|\boldsymbol{x}_i - \boldsymbol{c}_k\|^2$$

- Sum over all points
- Sum over all clusters
- Point $\boldsymbol{x}_i$ is assigned to cluster k
- Center of cluster k

K-Means Clustering

- We want to find K cluster centers and an assignment of points to cluster centers to minimize the sum of squared Euclidean distances between each point and its assigned cluster center:

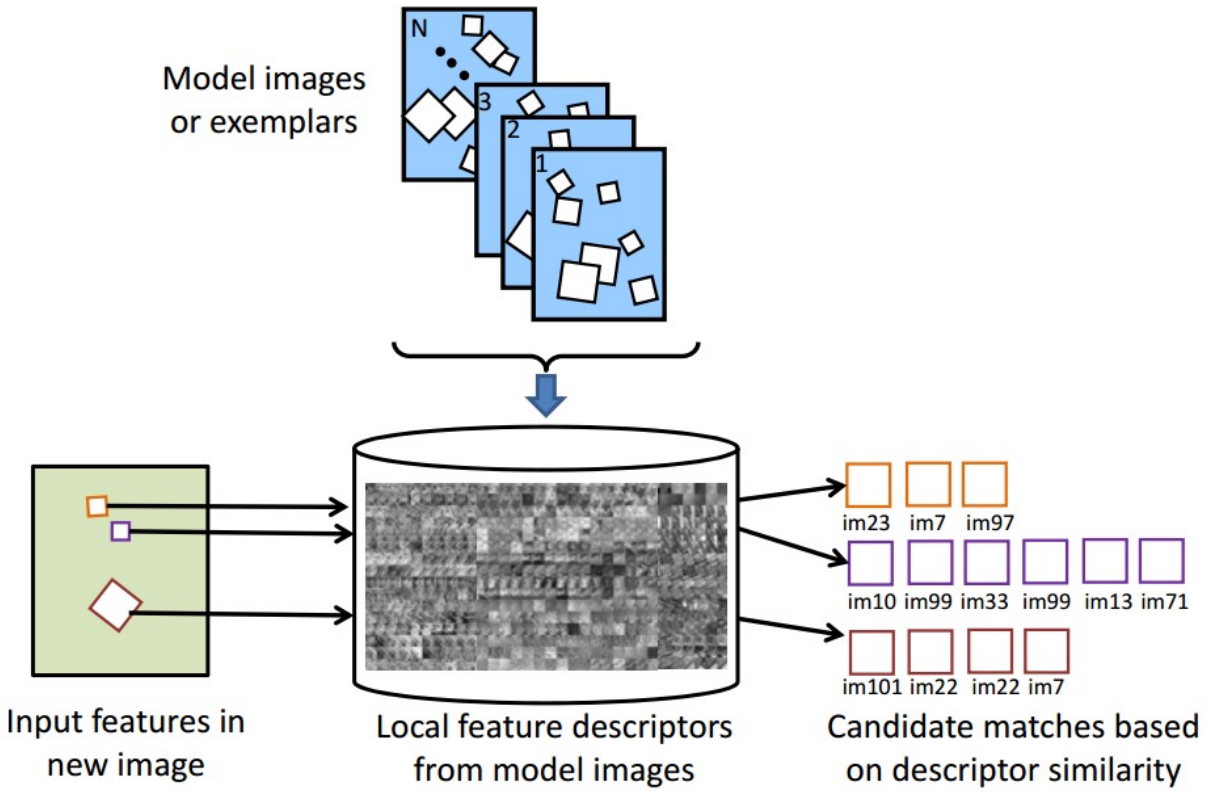
$$\sum_i \sum_k a_{ik} \|\boldsymbol{x}_i - \boldsymbol{c}_k\|^2$$

- Algorithm:
  - Randomly initialize K cluster centers
  - Iterate until convergence:
    - Assign each data point to its nearest center
    - Recompute each cluster center as the mean of all points assigned to it

K-Means Example

Source

How to Do the Indexing?



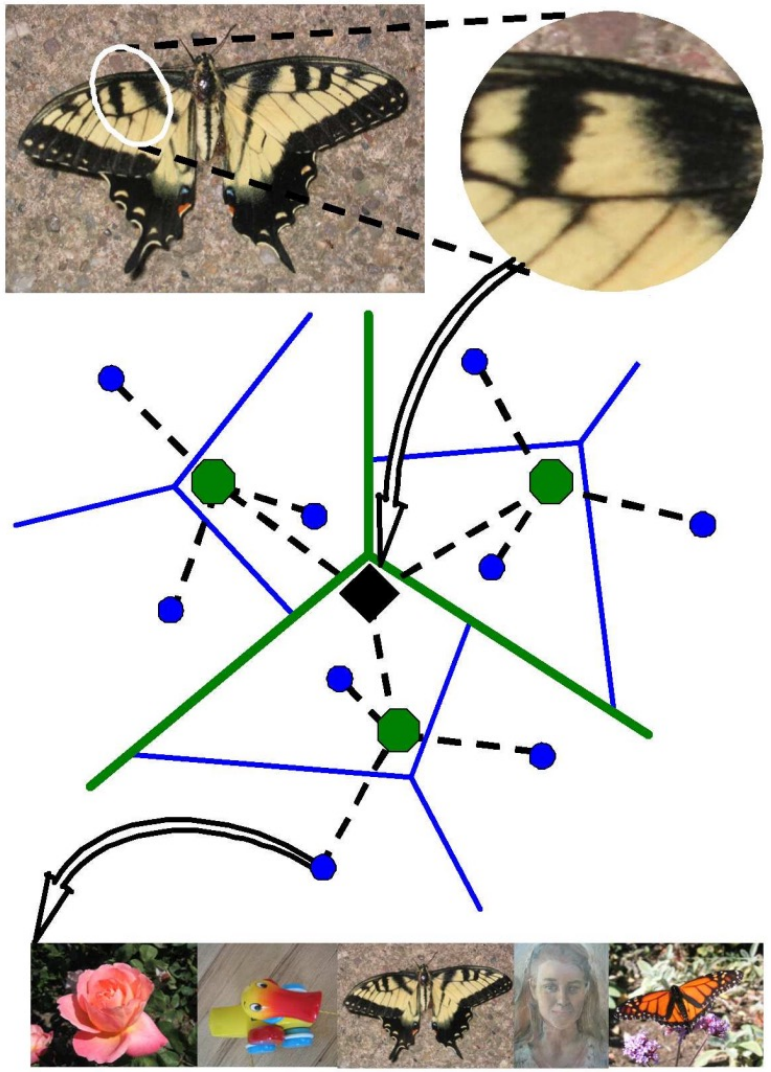
- Cluster descriptors in the database to form codebook
- At query time, quantize descriptors in query image to nearest codevectors
- Problem solved?

Efficient Indexing Technique: Vocabulary Trees

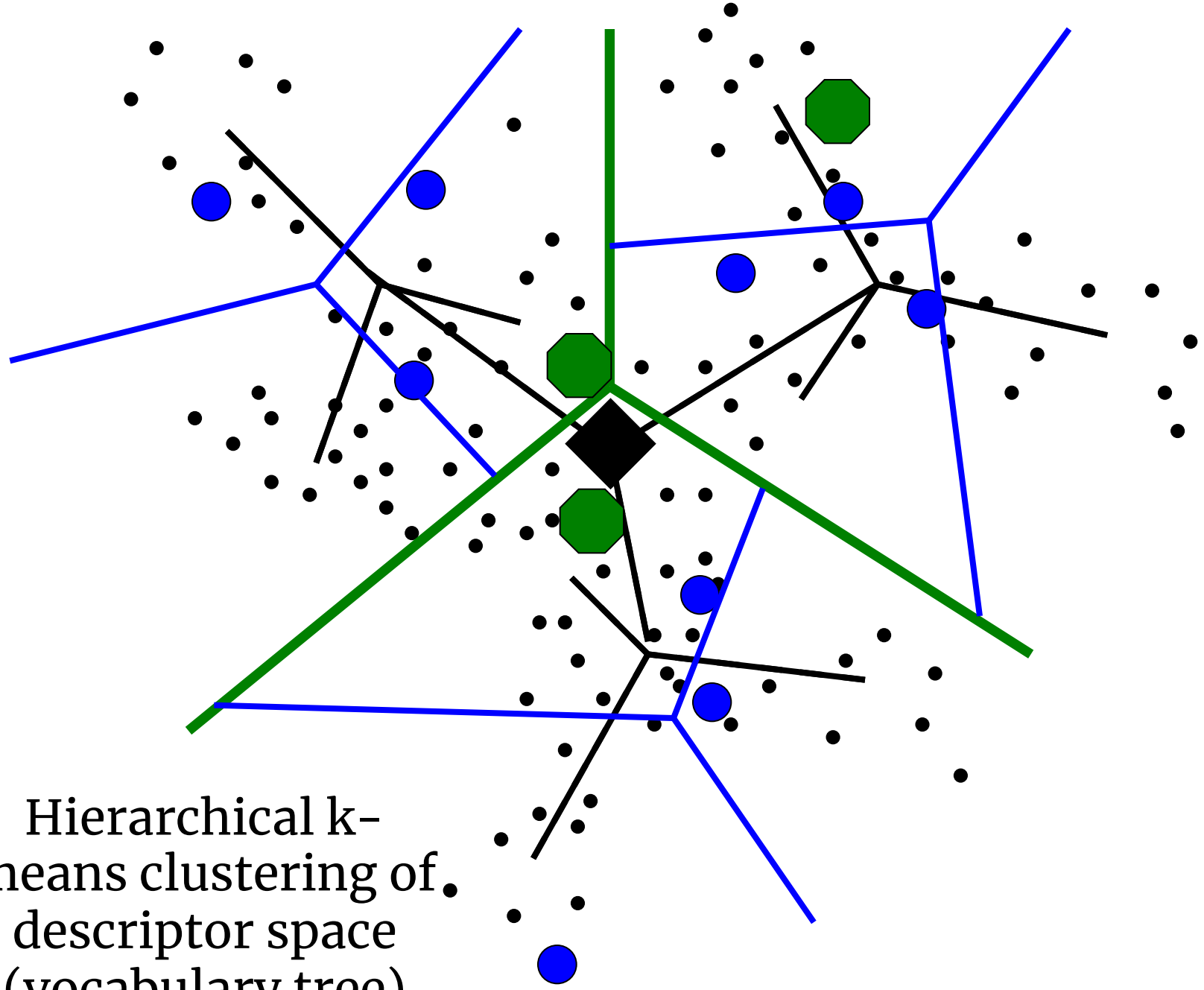
Test image

Vocabulary tree with inverted index

Database

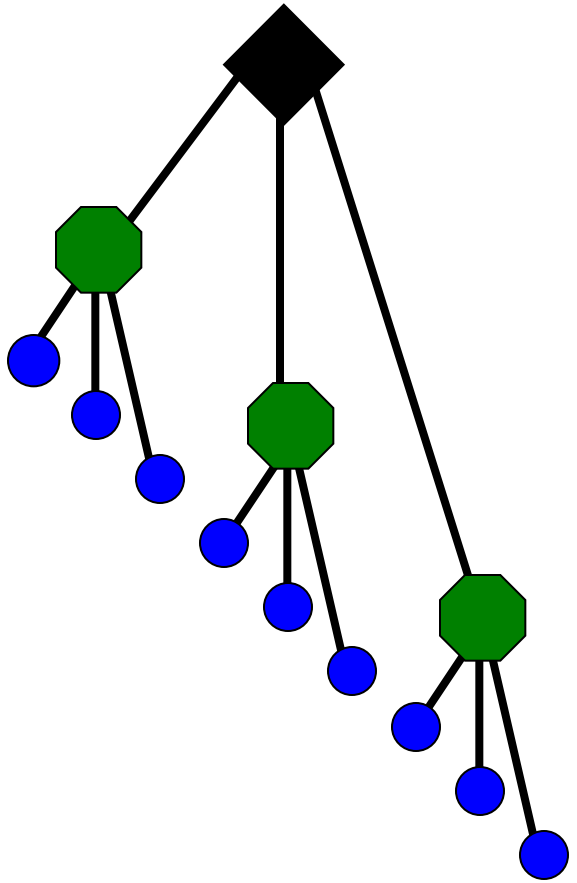


D. Nistér and H. Stewénius, Scalable Recognition with a Vocabulary Tree, CVPR 2006



Hierarchical k-means clustering of descriptor space (vocabulary tree)

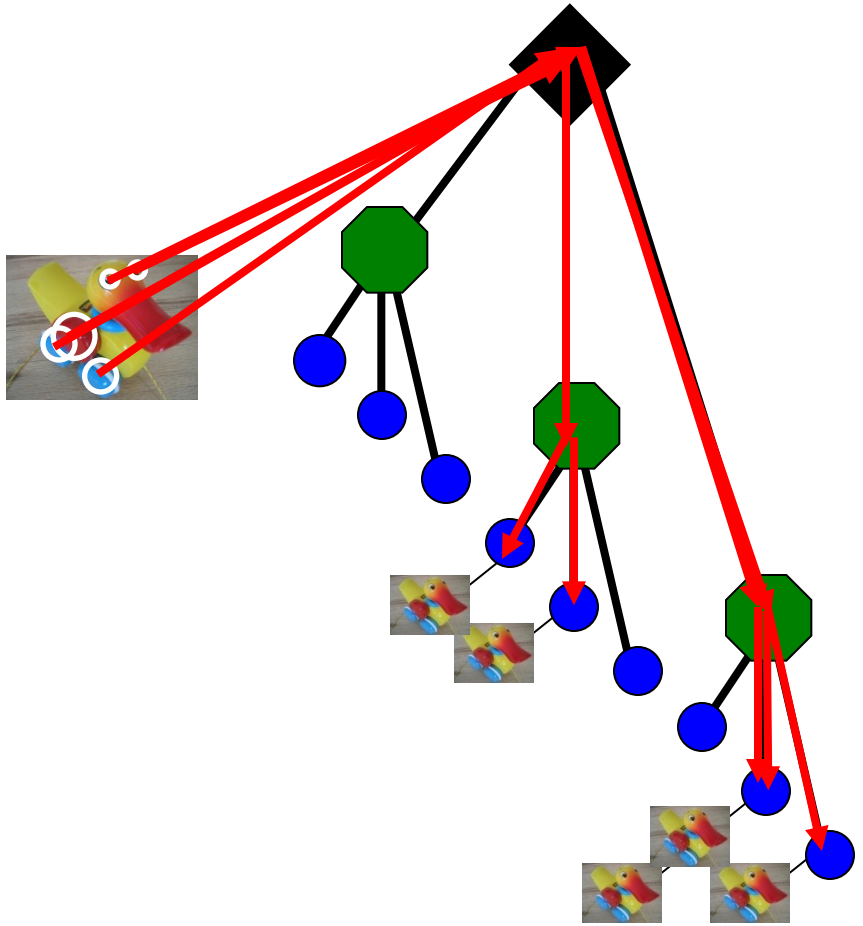
Slide credit: D. Nister



Vocabulary tree/inverted index

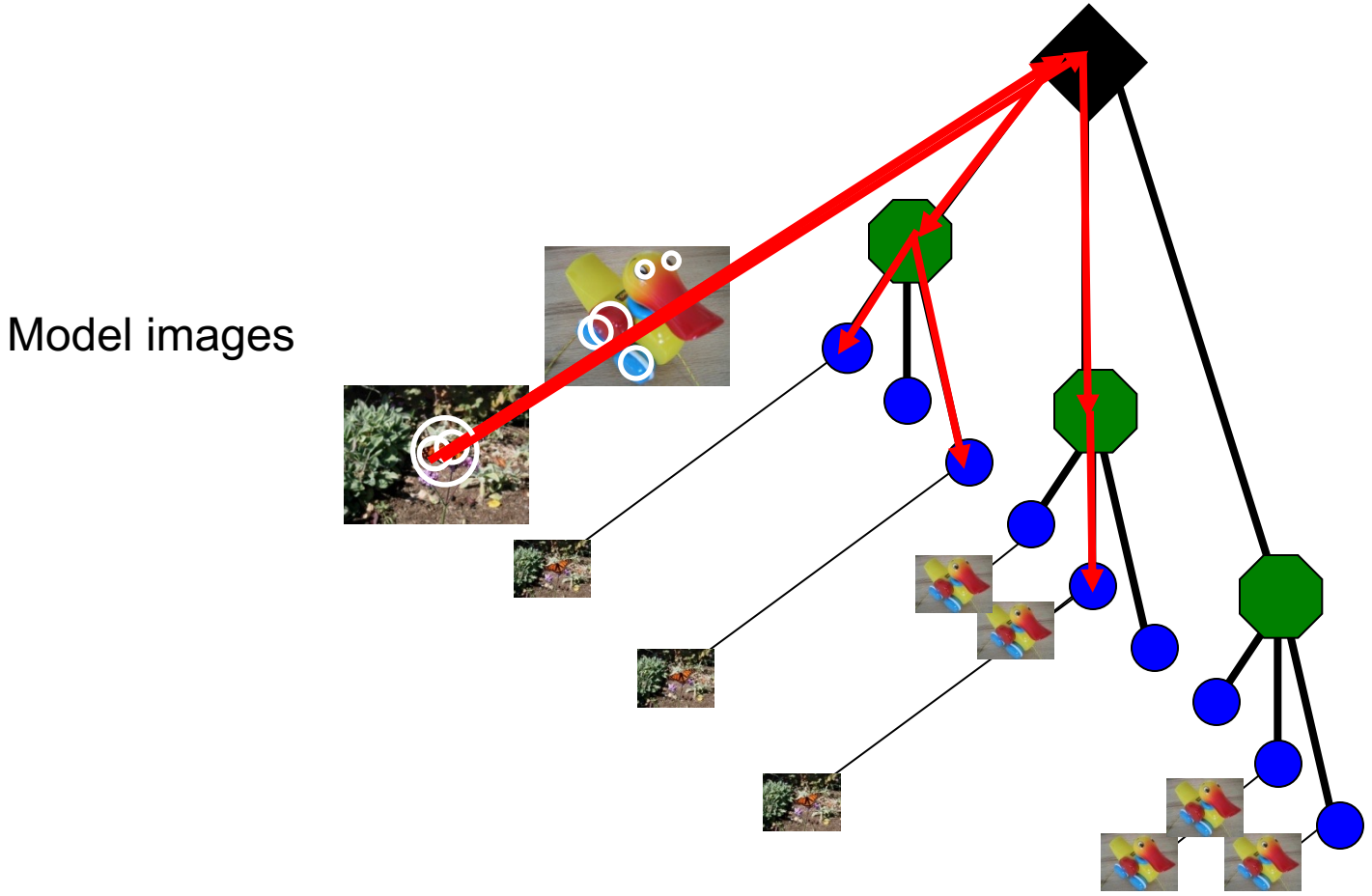
Slide credit: D. Nister

Model images



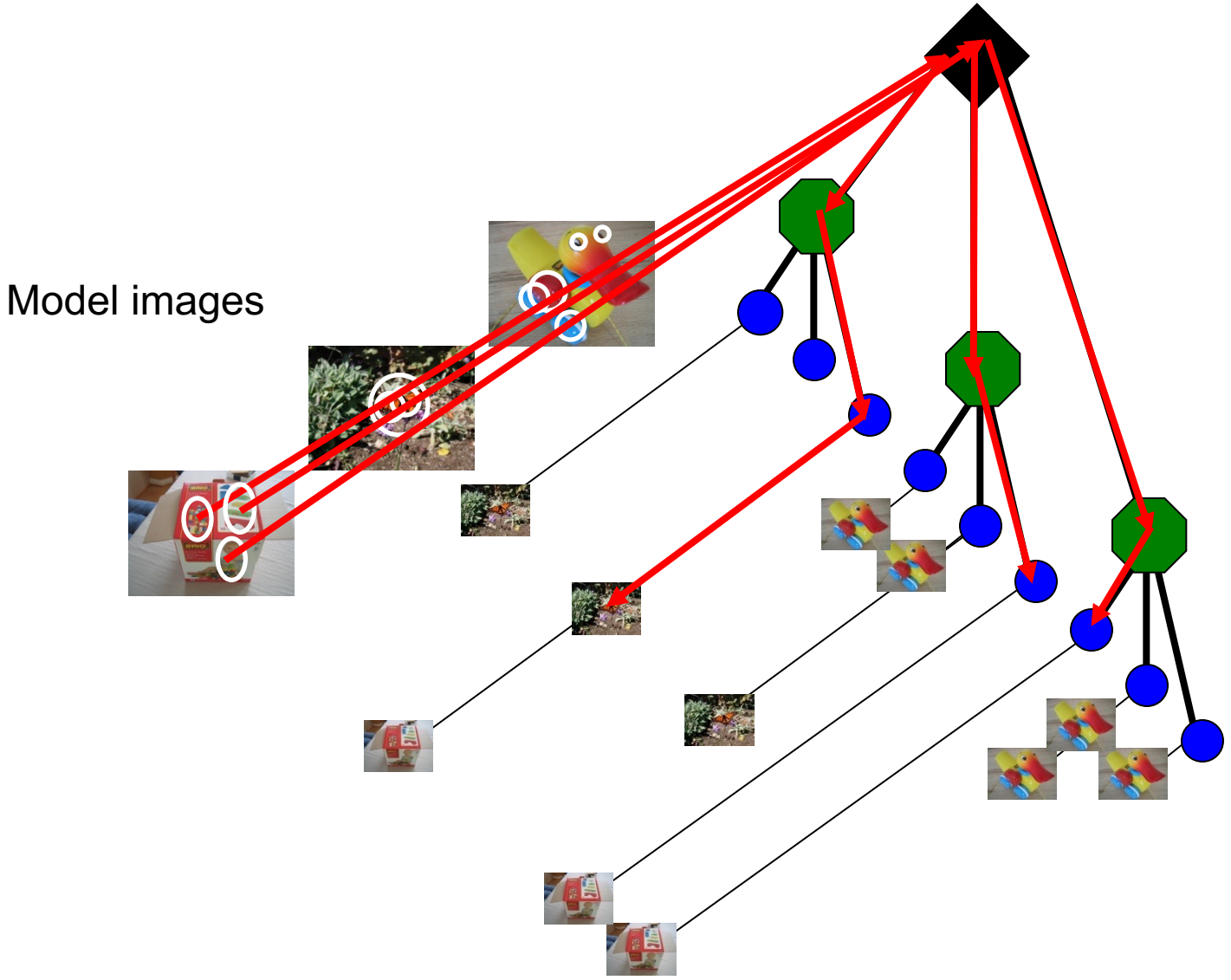
Populating the vocabulary tree/inverted index

Slide credit: D. Nister



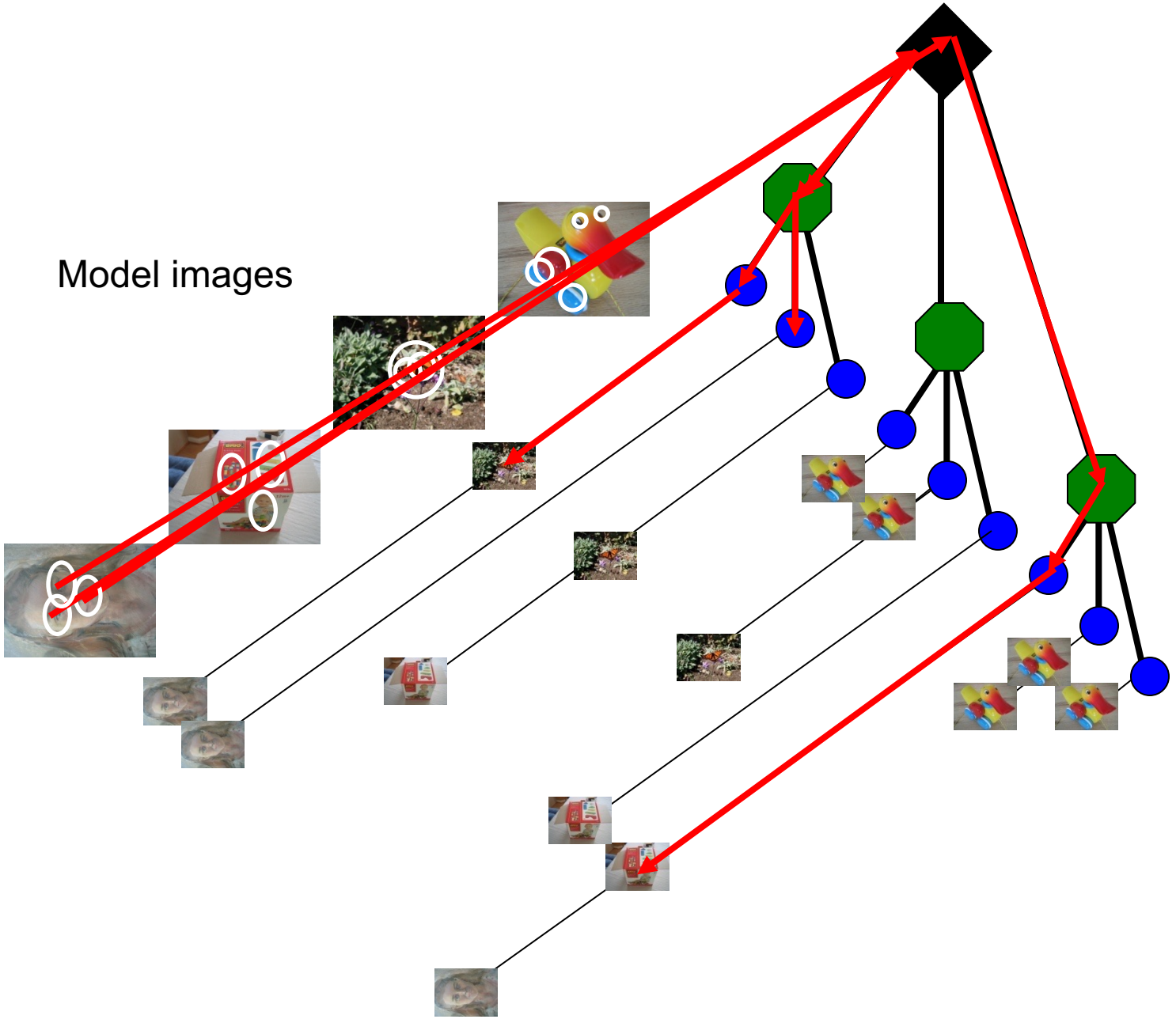
Populating the vocabulary tree/inverted index

Slide credit: D. Nister



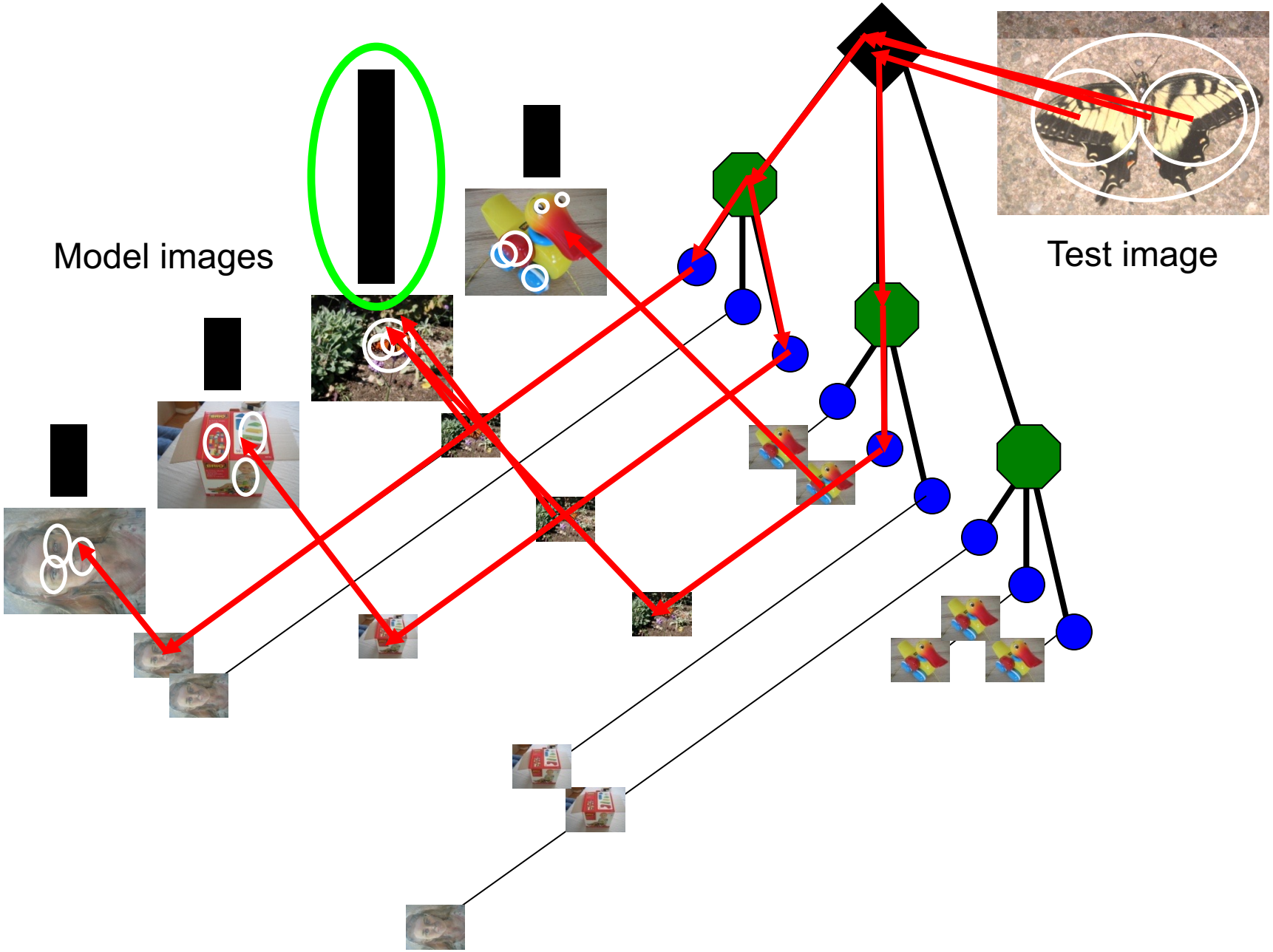
Populating the vocabulary tree/inverted index

Slide credit: D. Nister



Populating the vocabulary tree/inverted index

Slide credit: D. Nister



Looking up a test image

Slide credit: D. Nister